PROGRAMAÇÃO PARA A INTERNET 2 (PROGWEB 2)

Caderno de Roteiros Práticos da Unidade Curricular

Curso Superior de Tecnologia em Sistemas para a Internet — IFSC Garopaba

Sumário

1 Visão Geral da Unidade Curricular **3**

2 Roteiro Prático 01 — Retomada e Transição Parte 1: Bootstrap 5 & Layout Responsivo Estático **5**

2.1 Introdução, Site Oficial e Inclusão do Bootstrap 5 6
2.2 Detalhamento Didático dos Exemplos de Bootstrap 5 (Exemplos 1 a 5) 6
2.3 Guia de Apoio: ”Para Explorar Mais no Bootstrap 5” 7
2.4 Tarefa Prática de Laboratório (Construção do Layout Estático) 8
2.5 Checklist Final da Aula 1 8

3 Roteiro Prático 02 — Retomada e Transição Parte 2: jQuery, Manipulação do DOM & LocalStorage **9**

3.1 Introdução, Sites Oficiais e Organização em Arquivo Externo `app.js` 10
3.2 Conceitos Fundamentais: O que é o DOM (Document Object Model)? 10
3.3 Sintaxe Básica do jQuery & O Comando `$(document).ready()` 11
3.4 Metodologia Pedagógica Incremental: A Mesma Aplicação Base, Recursos Crescentes . . 11
3.5 Detalhamento Didático da Estrutura HTML Base Unificada & Exemplos 6 a 11 12
3.6 Guia de Apoio: ”Para Explorar Mais no jQuery” 13
3.7 Parte 2 — Tarefa Prática: Programação Dinâmica em Arquivo Externo app.js 14
3.8 Parte 3 — Desafio Extra: Filtro Dinâmico em Tempo Real 14
3.9 Checklist Final da Aula 2 15

4 Roteiro Prático 03 — Atividade Prática Autônoma: Calculadora de Pedidos & Carrinho de Compras **16**

4.1 Introdução e Contextualização da Atividade 17
4.2 Fundamentação Teórica: Como Ler e Calcular Valores de Campos com jQuery 17
4.2.1 1. Leitura e Conversão de Texto para Número (`parseFloat` e `parseInt`) 17
4.2.2 2. Soma de Múltiplos Checkboxes Marcados com `.each()` 18
4.2.3 3. Guia de Eventos do Navegador (`input`, `change` e `click`) 18

4.2.4 4. Formatação Financeira em Real (R$) 19
4.3 Dicas de Estruturação Visual & Componentes do Bootstrap 5 19
4.4 Roteiro Guiado Passo a Passo (Roteiro 3) 19
4.5 Checklist de Conclusão e Orientações de Entrega 22

5 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas 23
5.1 Introdução e Contextualização do Projeto 24
5.2 Estrutura Arquitetural de Pastas 24
5.3 Fundamentação Teórica: Componentes Modernos de Interface (Bootstrap 5) 25
5.3.1 1. Janelas Modais (*Modals*): Diálogos Sobrepostos de Alta Atenção 25
5.3.2 2. Notificações Flutuantes (*Toasts*): Feedback Assíncrono Não-Intrusivo 26
5.3.3 3. Cartões de Conteúdo (*Cards*): Unidades Modulares e Grid Uniforme 26
5.3.4 4. Validação Visual e o Campo Oculto (`type="hidden"`) 27
5.4 Fundamentação Teórica: Arquitetura Modular com ES6 Modules 28
5.4.1 1. Declaração no HTML (`type="module"`) 28
5.4.2 2. Exportação e Importação (`export` e `import`) 28
5.5 Passo a Passo Guiado do Roteiro 30
5.6 Checklist de Conclusão e Critérios Avaliativos 46

6 Roteiro Prático 05 — Assincronismo, Fetch API & Backend RESTful em Flask (SQLite) 47
6.1 Introdução: Da Vitrine Isolada à Aplicação em Rede 48
6.2 Visão Geral da Arquitetura Cliente-Servidor 48
6.3 Estrutura de Pastas do Projeto Integrado 49
6.4 Conceitos Fundamentais em Pílulas Práticas de Aquecimento 49
6.4.1 Pílula 1: Assincronismo e Leitura de Dados com Fetch API (GET) 49
6.4.2 Pílula 2: Microbackend Flask e a Política de Mesma Origem (CORS) 52
6.4.3 Pílula 3: Envio de Dados via POST (JSON.stringify e Headers HTTP) 54
6.4.4 Pílula 4: Resiliência de Rede e Feedback Visual (try/catch/finally com Spinner) . . 57
6.5 Implementação Passo a Passo Guiada 60
6.6 Quadro Comparativo: Evolução do Roteiro 04 para o Roteiro 05 72

1 Visão Geral da Unidade Curricular

A Unidade Curricular de **Programação para a Internet II** tem como objetivo aprofundar os conhecimentos adquiridos na disciplina antecedente (Programação para a Internet I), evoluindo o desenvolvimento frontend básico em HTML5, CSS3 e JavaScript Vanilla para a construção de **Aplicações Web Modernas, Reativas, Modulares e Integradas a Serviços Web RESTful com aplicação de extensão comunitária**.

Ao longo do semestre, em encontros semanais de **3h/aula em laboratório**, os estudantes irão desenvolver competências avançadas em arquitetura de código client-side, modularização com ES6 Modules, manipulação avançada de estado no cliente, consumo de APIs assíncronas, roteamento no cliente (Single Page Applications — SPAs), introdução a ferramentas modernas de build (Vite) e desenvolvimento de um Projeto Prático Integrador de Extensão.

Estrutura do Semestre

O semestre está organizado em 5 módulos estruturantes de aprendizagem prática:

1. **Retomada e Diagnóstico (Revisão de PI-I)**
   Revisão prática de manipulação do DOM, elementos visuais (Bootstrap 5), jQuery, assincronismo com `async/await`, Fetch API e armazenamento com `localStorage`.

2. **Arquitetura Modular e ES6 Modules**
   Organização profissional de código frontend, separação de responsabilidades em serviços, componentes e manipuladores de eventos.

3. **Gestão Avançada de Estado e Formulários Reativos**
   Persistência no navegador, validações avançadas no cliente, tratamento rigoroso de erros e experiência do usuário (UX).

4. **Consumo de APIs RESTful e Desenvolvimento de SPAs**
   Construção de Single Page Applications com roteamento client-side sem recarregamento de página e integração com rotas de backend.

5. **Introdução a Ecossistemas Modernos e Projeto de Extensão**
   Uso de bundlers (`Vite`), introdução à componentização reativa e entrega de um Projeto Prático Integrador de Extensão completo de final de semestre.

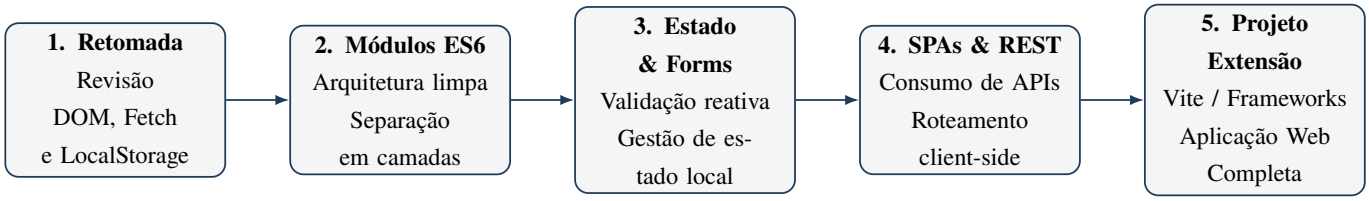


Figura 1: Pipeline de aprendizagem da UC Programação para a Internet II.

Competências Desenvolvidas

Ao final da disciplina, o estudante deverá ser capaz de:

- Construir interfaces web altamente dinâmicas e responsivas;
- Estruturar aplicações JavaScript utilizando arquitetura modular em ES6 Modules;
- Consumir e manipular dados de serviços Web RESTful usando `fetch` e `async/await`;
- Gerenciar estado local e persistência no cliente via Web Storage API;
- Implementar padrões de projeto de Single Page Application (SPA);
- Utilizar ferramentas modernas de ambiente de desenvolvimento web (`Node.js`, `npm`, `Vite`);
- Desenvolver um projeto prático integrador de extensão atendendo demandas reais da comunidade;
- Publicar e versionar aplicações web profissionais no GitHub e GitHub Pages.

Metodologia de Avaliação

A avaliação é contínua e formativa, dividida em três eixos principais:

1. **Avaliação Teórica Única (A1 — 25%):** Prova teórica individual de verificação dos conceitos fundamentais de arquitetura modular, assincronismo, protocolo HTTP, consumo de APIs e SPAs.
2. **Avaliação Contínua de Aulas (A2 — 35%):** Conjunto de **Participação Ativa + Pontualidade de Entrega** dos roteiros práticos semanais desenvolvidos nos laboratórios.
3. **Trabalho Prático Integrador / Extensão (A3 — 40%):** Desenvolvendo uma aplicação web completa de extensão comunitária/acadêmica que integra todos os conteúdos ministrados na unidade curricular (Bootstrap 5, JS ES6 Modular, Fetch API, LocalStorage, Roteamento SPA e Vite), hospedada no GitHub Pages com defesa demonstrativa.

2 Roteiro Prático 01 — Retomada e Transição Parte 1: Bootstrap 5 & Layout Responsivo Estático

Objetivos

Objetivos da Aula (Parte 1 — Bootstrap 5)
Ao final desta aula o estudante será capaz de:

- Conectar a base de HTML5 semântico e CSS3 de **ProgWeb 1** com a evolução prática de **ProgWeb 2**;
- Compreender o alinhamento do semestre com o Projeto Pedagógico do Curso (**PPC** CONSUP 30/2020 — 120h);
- Localizar e obter a biblioteca no site oficial do **Bootstrap 5** (`https://getbootstrap.com`);
- Configurar o ambiente incluindo os arquivos de estilo CSS e Bootstrap Icons no `<head>` do documento HTML;
- Aplicar o sistema de **Grid Responsivo de 12 Colunas** do Bootstrap 5 (`.container`, `.row`, `.col-md-*`);
- Estilitar elementos visuais da interface (Exemplos 1 a 5: Grid, Formulários, Cards, Badges, Modais e Tabelas Zebradas);
- Estruturar a interface do **Layout Estático Completo da Aplicação Web** (Exemplo 5);
- Acompanhar o material através da apresentação de slides interativa da Parte 1 (`https://chameoandre.github.io/CST-Sistemas-para-internet-programacao-para-internet-2/retomada-de-atividades-pi2/parte-1-bootstrap/index.html`).

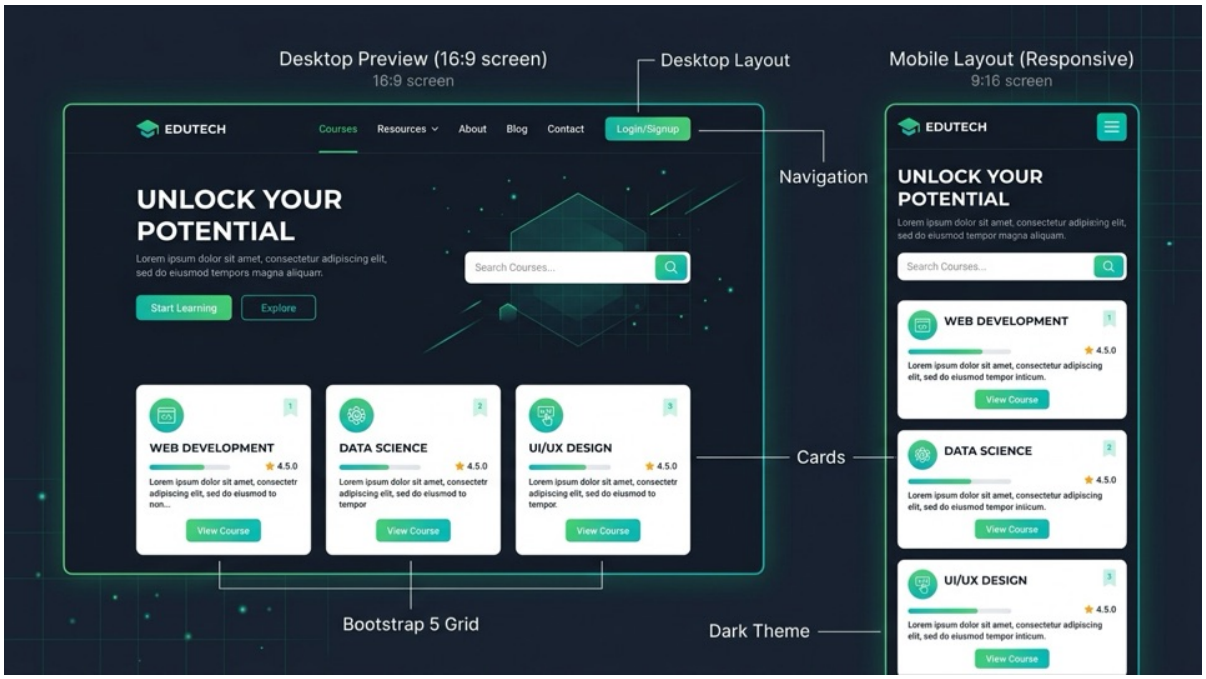


Figura 2: Visão Geral da Aplicação: Layout responsivo completo com Bootstrap 5 (Grid de 12 colunas, Formulários, Cards e Tabelas).

Introdução, Site Oficial e Inclusão do Bootstrap 5

Bem-vindos à primeira parte da retomada prática da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Neste primeiro encontro, focamos na construção da interface visual responsiva estática do nosso sistema utilizando o framework **Bootstrap 5**.

1. **Bootstrap 5 (Site Oficial & CDN):** No portal oficial `https://getbootstrap.com` e nas referências do W3Schools, obtemos a documentação de componentes e os links de CDN (`jsDelivr`) incluídos via tag `<link rel="stylesheet">` na seção `<head>`;

2. **Bootstrap Icons:** Inclusão do pacote de ícones vetoriais oficial do Bootstrap para ilustrar botões, cartões e alertas da aplicação;

3. **Acompanhamento Interativo:** O trabalho é guiado pelos slides da Parte 1 (`parte-1-bootstrap/index.html`) e pela prática na pasta `exercicio-retomada/`.

Detalhamento Didático dos Exemplos de Bootstrap 5 (Exemplos 1 a 5)

Abaixo está o detalhamento dos componentes de layout desenvolvidos nesta etapa:

- **Exemplo 1 (Bootstrap 5 — Grid Responsivo & Containers):** *Desenvolvimento de layout responsivo flexível.* A classe `.container` centraliza o conteúdo, `.row` cria a linha de grade e `.col-12 .col-md-6 .col-lg-4` ajusta a largura para 100% no celular, 50% em tablets e 33.3% em desktops;

- **Exemplo 2 (Bootstrap 5 — Formulários & Inputs Estilizados):** *Construção de campos com visual moderno.* A classe `.mb-3` adiciona margem inferior, `.form-label` formata o rótulo e `.form-control` estiliza o campo com bordas arredondadas e efeito de foco azul;
- **Exemplo 3 (Bootstrap 5 — Cards, Botões & Alertas Visuais):** *Empacotamento de componentes de interface.* Cria um cartão com elevação visual (`.card .shadow`), exibe um alerta verde de confirmação (`.alert .alert-success`) e insere um botão estilizado (`.btn .btn-primary`);
- **Exemplo 4 (Bootstrap 5 — Tabelas Elegantes & Utilitários):** *Formatação de tabelas com múltiplos registros.* Aplica `.table` para espaçamento automático, `.table-hover` para destacar a linha sob o mouse, `.table-striped` para alternar as cores das linhas (zebrado visual) e `.badge` para exibir tags coloridas de status;
- **Exemplo 5 (Bootstrap 5 — Layout Estático Completo):** *Layout integrado de cadastro e listagem.* Combina Grid responsivo (`.row .col-lg-4` e `.col-lg-8`), Formulário no Card e Tabela Zebrada em uma única interface completa, pronta para receber a interatividade na Parte 2.

Guia de Apoio: "Para Explorar Mais no Bootstrap 5"

Para dar suporte aos alunos na customização do layout, o quadro abaixo resume opções adicionais de classes do Bootstrap 5:

Opções Adicionais do Bootstrap 5 (Layout, Componentes e Utilitários)

Código / Classe	Descrição e Efeito Visual
`col-12 col-sm-6 col-lg-3`	Ocupa 100% no celular, 50% em tablets pequenos e 25% em desktops wide.
`form-select / form-control-lg`	Aplica o estilo de menu suspenso ou aumenta o tamanho do campo input.
`btn-outline-danger`	Botão vazado vermelho que preenche ao passar o ponteiro do mouse.
`shadow-sm / shadow-lg`	Aplica sombra suave ou pronunciada para destacar cartões da página.
`d-flex justify-content-between`	Ativa o Flexbox e alinha itens nas extremidades da caixa.

Tarefa Prática de Laboratório (Construção do Layout Estático)

Atividade Prática – Layout Estático com Bootstrap 5

Na pasta `exercicio-retomada/`, os estudantes deverão construir a estrutura estática:

1. **Inclusão dos Links no Head:** Abrir o arquivo `index.html` e incluir os links de CDN do Bootstrap 5 CSS e Bootstrap Icons;
2. **Montagem da Grade Responsiva:** Criar a estrutura do Grid dividindo a página entre a coluna de cadastro (`col-md-4`) e a coluna da tabela (`col-md-8`);
3. **Estilização de Formulários e Tabelas:** Aplicar as classes `.card`, `.form-control`, `.btn-primary` e `.table-striped` construindo o leiaute profissional completo (Exemplo 5).

Checklist Final da Aula 1

Verificação Técnica Parte 1

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

As dependências do Bootstrap 5 CSS e Icons estão corretamente vinculadas no `<head>`;

A página ajusta seu layout de forma fluida ao alterar a largura da janela do navegador;

A interface estática da aplicação (Exemplo 5) está completamente montada e legível.

3 Roteiro Prático 02 — Retomada e Transição Parte 2: jQuery, Manipulação do DOM & LocalStorage

Objetivos

Objetivos da Aula (Parte 2 — jQuery & Dinâmica Frontend)
Ao final desta aula o estudante será capaz de:

- Retomar o layout estático construído no Bootstrap 5 e adicionar inteligência no lado do cliente com **jQuery**;
- Localizar e obter a biblioteca no portal oficial do **jQuery** (`https://jquery.com` e `https://code.jquery.com`);
- Incluir os scripts no final do `<body>` e organizar a lógica em arquivo JavaScript externo (`app.js`);
- Compreender o manipulador de inicialização segura `$(document).ready(function(){...});`
- Selecionar elementos do DOM com `$()` e escutar eventos de formulário e clique (`.on('submit')`, `.click()`);
- Aplicar efeitos visuais e animações fluidas na interface (`.fadeIn()`, `.slideToggle()`, `.addClass()`);
- Inserir e remover elementos dinamicamente na árvore DOM com `.append()`, `.val()` e `.remove()`;
- Persistir e recuperar dados no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Implementar filtro de busca dinâmico em tempo real na tabela;
- Acompanhar o material através da apresentação de slides interativa da Parte 2 (`https://chameoandre.github.io/CST-Sistemas-para-internet-programacao-para-internet-2/retomada-de-atividades-pi2/parte-2-jquery/index.html`).

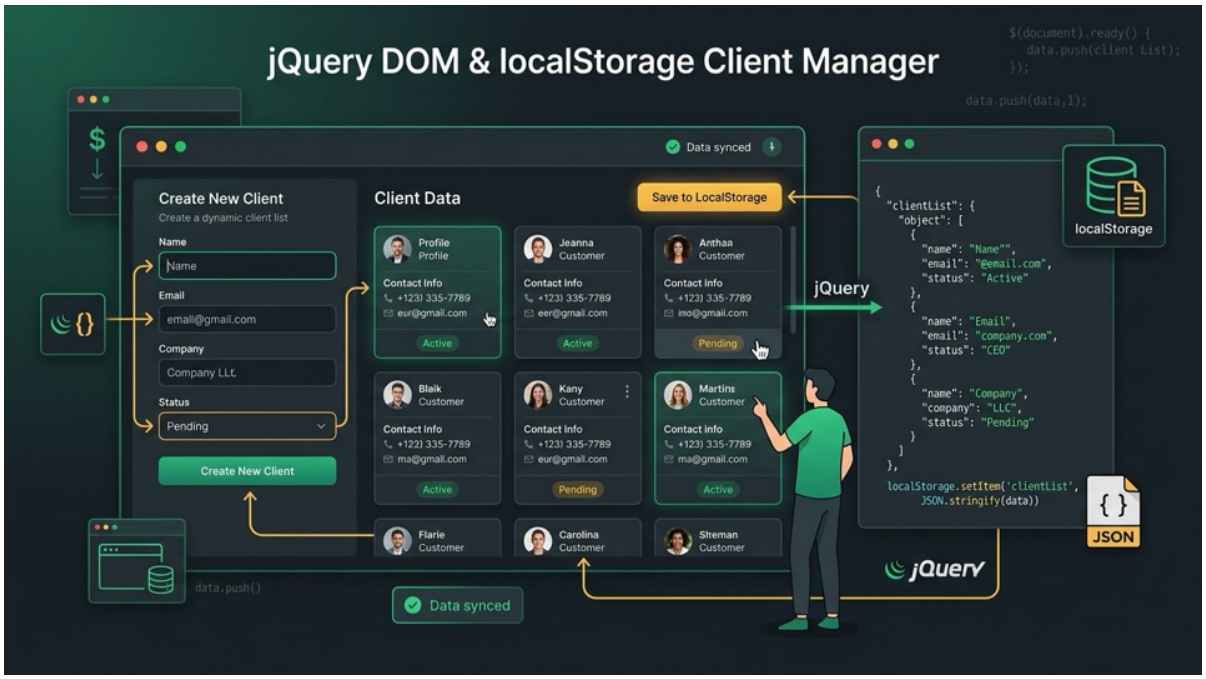


Figura 3: Visão Geral da Aplicação: Interface dinâmica client-side com jQuery, manipulação do DOM e persistência via LocalStorage.

Introdução, Sites Oficiais e Organização em Arquivo Externo `app.js`

Bem-vindos à segunda parte da retomada prática da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Nesta aula, transformamos a interface estática do Bootstrap 5 em um sistema **dinâmico, interativo e funcional** utilizando a biblioteca **jQuery**.

1. **jQuery (Site Oficial & CDN):** No portal `https://jquery.com` e no servidor de CDN `https://code.jquery.com`, obtemos os links da versão minificada (`jquery-3.7.1.min.js`), incluída via tag `<script>` antes do fechamento do `</body>`;

2. **Organização Profissional em Arquivo Externo (`app.js`):** Adotamos como boa prática o desacoplamento do código JavaScript em um arquivo separado (`app.js`), inicializado com `$(document).ready()`;

3. **Acompanhamento Interativo:** O trabalho é guiado pelos slides da Parte 2 (`parte-2-jquery/index.html`) e pela prática na pasta `exercicio-retomada/`.

Conceitos Fundamentais: O que é o DOM (Document Object Model)?

Antes de manipularmos os elementos da página com o jQuery, é fundamental compreender o papel do **DOM (Document Object Model)**:

- **Significado da Sigla:** *Document Object Model* (Modelo de Objeto do Documento). É a representação em memória (estrutura em árvore) que o navegador cria ao ler o arquivo HTML, transformando cada tag em um objeto manipulável;
- **Responsabilidades Principais do DOM:**
  1. **Mapeamento Hierárquico:** Organiza a relação estrutural entre elementos pais e filhos (ex: `<form>` contendo `<input>`);
  2. **Interface de Interatividade:** Fornece comandos para ler valores, alterar textos, aplicar estilos CSS e escutar eventos do usuário (cliques e submissões);
- **Interação via JavaScript / jQuery:** O código JS consulta a árvore usando seletores como `$('#input-nome')`. A partir daí, executa operações dinâmicas: lê dados (`.val()`), altera textos (`.html()`), modifica cores (`.css()`) e reage a eventos (`.on('submit')`).

Sintaxe Básica do jQuery & O Comando `$(document).ready()`

Antes de manipularmos o formulário, precisamos compreender como as instruções são estruturadas e por que o método de inicialização é indispensável:

- **A Sintaxe Padrão `$(seletor).acao()`:**
  - `$`: É o atalho para invocar a biblioteca jQuery;
  - `seletor`: A string com o seletor CSS do elemento desejado (ex: `'#btn-salvar'`);
  - `acao()`: O método que lê, modifica ou escuta o elemento (ex: `.val()`, `.html()`, `.on()`);
- **O que faz o `$(document).ready(function() {...})`:** É a função de **inicialização segura**. Ela garante que qualquer código JavaScript/jQuery só seja executado **após o navegador ter concluído a leitura do HTML e a montagem da árvore do DOM na memória RAM**;
- **Por que temos que utilizá-lo (A causa do erro):** Se o script rodar antes de o HTML ser lido, o elemento (ex: `#form-cadastro`) ainda não existirá na memória. O jQuery retornará um objeto nulo/vazio e qualquer tentativa de escutar um clique ou submissão falhará silenciosamente;
- **Sintaxe Abreviada Equivalente:** A comunidade também utiliza a forma curta moderna: `$(function() { /* código seguro */ });`.

Metodologia Pedagógica Incremental: A Mesma Aplicação Base, Recursos Crescentes

Para garantir que o aprendizado seja fluido e contínuo, adotamos a **Metodologia Incremental**:

- **Estrutura Base Unificada:** Em vez de criar um código HTML novo do zero para cada exercício, mantemos a mesma página de **Cadastro de Clientes** criada na Parte 1 com Bootstrap 5;

- **Acúmulo de Superpoderes Passo a Passo:**
  1. **Exemplo 6 (Base):** Leitura de inputs com `.val()` e escrita em alerta com `.html()`;
  2. **Exemplo 7 (Feedback):** Remoção de `.d-none`, `.addClass('alert-success')` e piscar animado 2x com `.fadeOut().fadeIn()`;
  3. **Exemplo 8 (Estilos):** Troca dinâmica de cores de fundo e bordas via `.css({ ... })`;
  4. **Exemplo 9 (Efeitos):** Deslizamento de painel retrátil com `.slideToggle()` e `.slideDown()`;
  5. **Exemplo 10 (DOM Dinâmico):** Montagem de tabela dinâmica com `.append()` e remoção de linha pai com `.closest('tr').remove()`;
  6. **Exemplo 11 (Persistência):** Armazenamento permanente no navegador com `localStorage` e conversão `JSON.stringify/parse`.

Detalhamento Didático da Estrutura HTML Base Unificada & Exemplos 6 a 11

Para manter uma linha contínua de aprendizado sem perder a conexão entre os seletores, todos os exemplos da Parte 2 utilizam a **mesma estrutura HTML base da aplicação de Cadastro de Clientes** (Exemplo 5 do Bootstrap 5), incrementando-a passo a passo:

Estrutura HTML Base Unificada (Formulário, Alerta e Tabela):

```
<!-- Formulário de Cadastro (#form-cadastro) -->
<form id="form-cadastro" class="card p-3 mb-3">
  <input type="text" id="input-nome"
         class="form-control mb-2" placeholder="Nome">
  <input type="email" id="input-email"
         class="form-control mb-2" placeholder="E-mail">
  <button type="submit" id="btn-salvar"
          class="btn btn-primary">Salvar Cliente</button>
</form>

<!-- Caixa de Alerta para Feedback Visual (#caixa-mensagem) -->
<div id="caixa-mensagem"
     class="alert alert-info d-none"></div>

<!-- Tabela com Corpo Dinâmico (#tabela-corpo) -->
<table class="table table-striped">
  <thead>
    <tr>
      <th>Nome</th>
      <th>E-mail</th>
      <th>Ação</th>
    </tr>
  </thead>
  <tbody id="tabela-corpo"></tbody>
</table>
```

Abaixo está a progressão funcional realizada sobre a mesma estrutura pedagógica:

- **Exemplo 6 (Passo 1 — Leitura & Escrita de Campos):** *Estrutura Base Inicial:* Captura a submissão do `#form-cadastro`, impede o recarregamento com `e.preventDefault()`, lê os valores de `#input-nome` e `#input-email` com `.val()` e escreve na caixa `#caixa-mensagem` com `.html()`;
- **Exemplo 7 (Passo 2 — Animações Visuais & Classes CSS):** *Adicionado em relação ao Exemplo 6:* Incrementa a `#caixa-mensagem` removendo a classe `.d-none`, aplicando `.addClass('alert-success')` e fazendo o alerta piscar/pulsar de forma altamente visível encadeando `.fadeOut(150).fadeIn(150).fadeO`
- **Exemplo 8 (Passo 3 — Troca Dinâmica de Cores e Estilos CSS):** *Adicionado em relação ao Exemplo 7:* Altera o estilo diretamente via código com `.css()`, modificando a cor de fundo do formulário e as bordas/cores da caixa de mensagem de forma dinâmica;
- **Exemplo 9 (Passo 4 — Efeitos Visuais de Deslizamento):** *Adicionado em relação ao Exemplo 8:* Requer a criação do botão `#btn-toggle-ajuda` e do painel retrátil `#painel-ajuda` (`style="display: none;"`). Explora o efeito `.slideToggle(300)` para alternar o painel de instruções e `.slideDown(400)` para a mensagem surgir deslizando;
- **Exemplo 10 (Passo 5 — Manipulação Dinâmica do DOM & Tabela):** *Adicionado em relação ao Exemplo 9:* Insere a nova linha `<tr>` com os dados e o botão de exclusão na `#tabela-corpo` com `.append()`. Ao clicar em Remover, exclui a linha pai com `$(this).closest('tr').fadeOut(300, function() { $(this).remove(); });`
- **Exemplo 11 (Passo 6 — Persistência no LocalStorage com JSON):** *Adicionado em relação ao Exemplo 10:* Salva o array de clientes no navegador com `localStorage.setItem('lista_clientes', JSON.stringify(clientes))` e restaura a tabela ao recarregar a página com `JSON.parse()`.

Guia de Apoio: ”Para Explorar Mais no jQuery”

Para dar suporte aos alunos na programação dinâmica, o quadro abaixo resume métodos essenciais do jQuery:

Opções Adicionais de jQuery (Seletores, Efeitos e Manipulação do DOM)

Método / Seletor	Descrição e Uso no Código
`$('.minha-classe')`	Seleciona todos os elementos do DOM que possuem uma classe CSS específica.
`$(el).slideDown() / .slideUp()`	Animação fluida de expandir para baixo ou recolher para cima.
`$(el).attr('disabled', true)`	Lê ou altera atributos HTML de um elemento (ex: desativar botões).
`$(el).prepend(html)`	Insere novo conteúdo HTML no início do elemento selecionado.
`$(el).empty() / .remove()`	Limpa todo o conteúdo interno ou remove o próprio elemento da página.

Parte 2 — Tarefa Prática: Programação Dinâmica em Arquivo Externo app.js

Atividade Prática – Dinâmica com jQuery e LocalStorage

Na pasta `exercicio-retomada/`, os estudantes deverão tornar a aplicação 100% dinâmica:

1. **Inclusão do jQuery:** Adicionar a tag `<script src="https://code.jquery.com/jquery-3.7.1.min.` antes do `</body>`;
2. **Seleção e Eventos em `app.js`:** No arquivo externo `app.js`, capturar a submissão do formulário com `$('#formCadastro').on('submit');`
3. **Inserção Dinâmica no DOM:** Ler os campos com `$('#nome').val()` e adicionar a nova linha na tabela com `$('#tabela-body').append();`
4. **Persistência no LocalStorage (Exemplo 9):** Salvar a lista de registros com `localStorage.setItem()` e tratar a restauração dos dados ao abrir a página (F5).

Parte 3 — Desafio Extra: Filtro Dinâmico em Tempo Real

Desafio Extra – Filtro por Nome com jQuery

Como atividade de aprimoramento em sala:

1. Criar um campo de busca por nome acima da tabela (`$('#filtro-nome')`);
2. Implementar o evento `input` do jQuery para filtrar dinamicamente as linhas exibidas à medida

que o usuário digita.

Checklist Final da Aula 2

Verificação Técnica Parte 2

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

- O script do jQuery 3.7.1 está incluído e o arquivo `app.js` inicializa com `$(document).ready();`
- Ao cadastrar um novo registro, a tabela é atualizada instantaneamente sem recarregar a página;
- Ao pressionar F5 (Recarregar), os dados salvos no `LocalStorage` permanecem visíveis na tela.

4 Roteiro Prático 03 — Atividade Prática Autônoma: Calculadora de Pedidos & Carrinho de Compras

Objetivos

Objetivos da Atividade (Atividade Guiada de 2 Semanas — Calculadora de Pedidos)
Ao final desta atividade prática autônoma, o estudante será capaz de:

- Consolidação prática e aplicação integrada do **Bootstrap 5** e do **jQuery** em uma aplicação web funcional;
- Ler valores de campos de formulário (`<select>`, `<input type="number">`, `<input type="checkbox">`) e convertê-los com segurança para tipos numéricos utilizando `parseFloat()` e `parseInt()`;
- Tratar valores vazios ou não numéricos (`NaN`) utilizando o operador de fallback `|| 0`;
- Compreender a diferença e aplicar os eventos corretos do navegador: `.on('input')` para digitação em tempo real, `.on('change')` para alteração de seleção/switches e `.on('click')` para botões;
- Realizar o cálculo automático do subtotal, taxa de entrega, descontos por cupom e total geral em tempo real, sem necessidade de recarregar a página;
- Formatar valores monetários para o padrão brasileiro (`R$ X,XX`) utilizando a função `.toFixed(2).replace('.', ',')`;
- Persistir e restaurar o rascunho do pedido no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Organizar o projeto em arquivo externo `app.js` e publicar a aplicação no **GitHub Pages**.

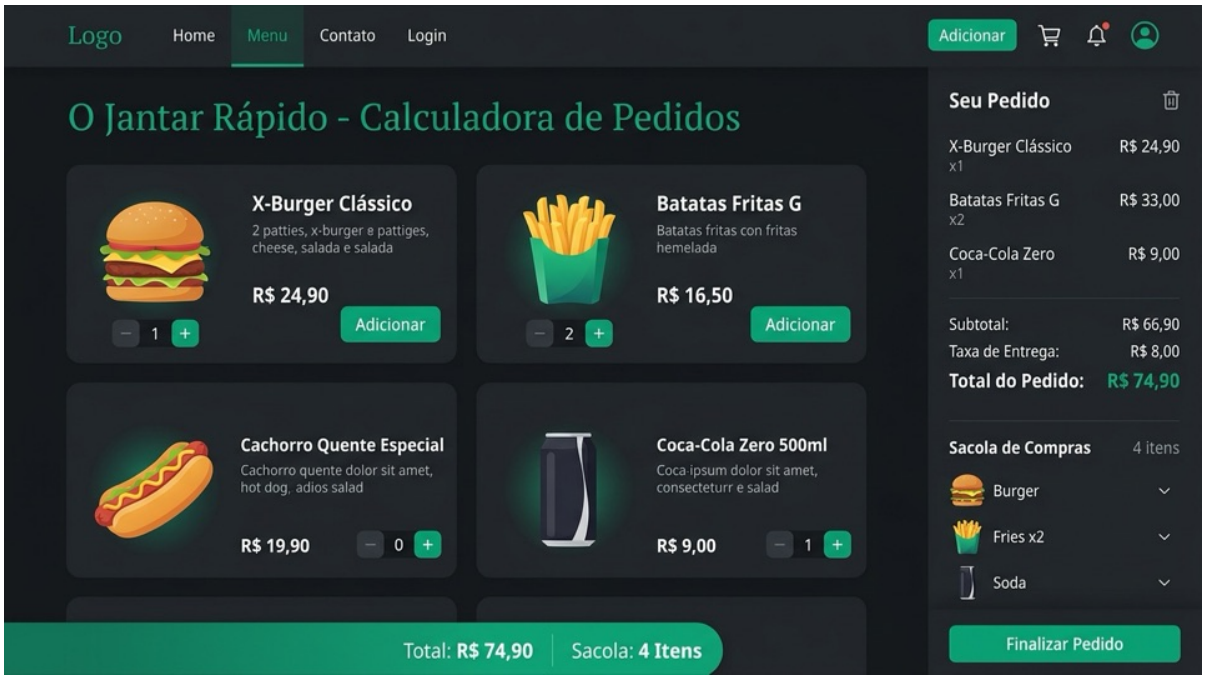


Figura 4: Visão Geral da Aplicação: Calculadora de Pedidos & Carrinho de Compras com cálculos em tempo real e cupons de desconto.

Introdução e Contextualização da Atividade

Prezados estudantes da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Durante as próximas 2 semanas, enquanto o professor estará em afastamento de saúde, vocês realizarão uma atividade prática autônoma guiada para consolidar os conceitos aprendidos sobre **Bootstrap 5**, **jQuery**, **Cálculos Numéricos no Cliente** e **Persistência no LocalStorage**.

O tema proposto é o desenvolvimento da **Lanchonete Express — Calculadora de Pedidos & Carrinho de Compras**. A aplicação permitirá que o cliente escolha seu lanche principal, selecione adicionais opcionais (queijo, bacon, batata frita, bebida), ajuste a quantidade, escolha o tipo de entrega, aplique um cupom de desconto e veja o **valor total a pagar atualizado instantaneamente na tela**.

Fundamentação Teórica: Como Ler e Calcular Valores de Campos com jQuery

Para realizar cálculos financeiros em uma página web, é necessário seguir três etapas indispensáveis:

4.2.1 1. Leitura e Conversão de Texto para Número (`parseFloat` e `parseInt`)

No HTML, todos os valores obtidos através de `$(seletor).val()` são retornados como **texto (string)**. Se tentarmos somar textos em JavaScript (ex: `"18.00" + "4.00"`), o resultado será uma concatenação (`"18.004.00"`), e não uma soma numérica.

Por isso, devemos converter o valor lido:

- `parseFloat(string)`: Converte o texto para número decimal (usado para preços);

- `parseInt(string)`: Converte o texto para número inteiro (usado para quantidades);
- **Tratamento de NaN (Not a Number):** Caso o campo esteja vazio, o `parseFloat()` retornará `NaN`. Para evitar que os cálculos quebrem, usamos a instrução `|| 0`.

Exemplo de leitura segura com fallback zero:

```
const precoLanche = parseFloat($('#select-lanche').val()) || 0;
const quantidade = parseInt($('#input-qtd').val()) || 1;
```

4.2.2 2. Soma de Múltiplos Checkboxes Marcados com `.each()`

Quando a aplicação possui múltiplos adicionais (ex: switches de queijo, bacon, batata frita), podemos percorrer apenas os checkboxes que o usuário marcou usando a combinação do seletor `:checked` com o método `.each()`:

Varredura de adicionais selecionados:

```
let somaAdicionais = 0;

$('.check-adicional:checked').each(function() {
  somaAdicionais += parseFloat($(this).val()) || 0;
});
```

4.2.3 3. Guia de Eventos do Navegador (`input`, `change` e `click`)

Para proporcionar uma excelente experiência de usuário (UX), a calculadora deve reagir instantaneamente a qualquer alteração realizada na tela. Escolha o evento adequado para cada tipo de elemento:

Guia de Eventos do Navegador em jQuery

Evento jQuery	Componente Indicado e Comportamento
`.on('change')`	Indicado para `select`, `checkbox` e `switch`. Disparado quando o usuário altera a seleção de um dropdown ou marca/desmarca uma caixa.
`.on('input')`	Indicado para `input type="number"` ou `text`. Disparado instantaneamente a cada caractere digitado ou alterado no campo.
`.on('click')`	Indicado para botões. Disparado quando o usuário clica nos botões de incremento (+/-), aplicar cupom ou finalizar pedido.

4.2.4 4. Formatação Financeira em Real (`R$`)

Para exibir valores numéricos formatados na moeda brasileira (`R$ 25,50`), crie uma função auxiliar utilizando `.toFixed(2)` para fixar duas casas decimais e `.replace('.', ',')` para trocar o ponto pela vírgula:

Função de Formatação de Moeda Brasileira:

```
function formatarMoeda(valor) {
  return 'R$ ' + valor.toFixed(2).replace('.', ',');
}

// Atualização da interface gráfica:
$('#total-geral').text(formatarMoeda(totalCalculado));
```

Dicas de Estruturação Visual & Componentes do Bootstrap 5

Para que a aplicação tenha um visual profissional e limpo, recomendamos a utilização dos seguintes painéis e utilitários do **Bootstrap 5**:

1. **Grid de 2 Colunas (`.row` e `.col-lg-7` / `.col-lg-5`):** Divida a tela em dois painéis principais — a coluna da esquerda para o formulário de montagem do pedido e a coluna da direita para o resumo financeiro;
2. **Painel de Montagem (`.card` e `.card-body`):** Agrupe as opções do pedido em um cartão elevado com bordas suaves;
3. **Adicionais com Switches (`.form-check .form-switch`):** Utilize interruptores modernos para os adicionais opcionais (queijo, bacon, fritas), incluindo a etiqueta `.badge` com o preço;
4. **Input Groups para Quantidade (`.input-group`):** Combine os botões de decremento (`-`) e incremento (`+`) com o campo numérico no meio;
5. **Resumo Financeiro em Lista (`.list-group .list-group-flush`):** Exiba a discriminação do subtotal, taxa de entrega e desconto em uma lista estilizada;
6. **Destaque do Total Geral (`.alert` ou caixa personalizada):** Destaque o valor total a pagar com tamanho de fonte ampliado e fundo contrastante.

Roteiro Guiado Passo a Passo (Roteiro 3)

Siga os passos abaixo para construir a aplicação na sua pasta de exercícios (`roteiro-3/exercicio-roteiro3/`):

Passo 1: Estrutura HTML5 e Inclusão das CDNs

Crie o arquivo `index.html` e inclua as bibliotecas do **Bootstrap 5**, **Bootstrap Icons** e **jQuery 3.7.1** via CDN antes de fechar a tag `</body>`.

```
<!-- Bootstrap 5 CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<!-- Arquivo da sua lógica JS -->
<script src="app.js"></script>
```

Passo 2: Formulário de Montagem do Pedido

Crie o menu de seleção do lanche principal atribuindo os preços no atributo `value` de cada `<option>`.

```
<select class="form-select" id="select-lanche">
  <option value="0">-- Selecione um lanche --</option>
  <option value="18.00">X-Burger Clássico - R$ 18,00</option>
  <option value="22.00">X-Salada Especial - R$ 22,00</option>
  <option value="26.00">X-Bacon Supremo - R$ 26,00</option>
</select>
```

Passo 3: Lógica de Cálculo no app.js

No arquivo `app.js`, estruture a função `$(document).ready()` e crie a função principal `calcularTotal()`:

```
$(document).ready(function() {

  function calcularTotal() {
    const precoLanche = parseFloat($('#select-lanche').val()) || 0;

    let somaAdicionais = 0;
    $('.check-adicional:checked').each(function() {
      somaAdicionais += parseFloat($(this).val()) || 0;
    });

    let qtd = parseInt($('#input-qtd').val()) || 1;
    const subtotal = (precoLanche + somaAdicionais) * qtd;
    const taxaEntrega = parseFloat($('#select-entrega').val()) || 0;

    const totalGeral = subtotal + taxaEntrega;

    $('#total-geral').text('R$ ' + totalGeral.toFixed(2).replace('.', ','));
  }

  // Escutadores em tempo real
  $('#select-lanche, #select-entrega, .check-adicional').on('change', calcularTotal);
  $('#input-qtd').on('input change', calcularTotal);

});
```

Passo 4: Persistência no LocalStorage

Implemente o salvamento automático do rascunho do pedido no `localStorage` ao clicar em "Finalizar Pedido" e a restauração automática ao pressionar `F5`:

```
// Salvar no LocalStorage
$('#btn-finalizar').on('click', function() {
  const pedido = {
    lanche: $('#select-lanche').val(),
    qtd: $('#input-qtd').val(),
    entrega: $('#select-entrega').val()
  };
  localStorage.setItem('rascunho_pedido', JSON.stringify(pedido));
  alert('Pedido salvo no navegador!');
});
```

Checklist de Conclusão e Orientações de Entrega

Antes de enviar sua atividade, verifique se você concluiu todos os itens da lista abaixo:

- ☐ A página carrega corretamente sem erros no console do navegador (`F12`);
- ☐ O layout é responsivo e utiliza os cartões e formulários do Bootstrap 5;
- ☐ O cálculo do valor total reage em tempo real a qualquer mudança nos selects, checkboxes ou quantidade;
- ☐ Os valores numéricos são devidamente formatados em Reais (`R$ X,XX`);
- ☐ O rascunho do pedido é salvo e restaurado via `localStorage`;
- ☐ O código foi commitado no repositório Git pessoal e publicado via **GitHub Pages**.

5 Roteiro Prático 04 — Arquitetura Frontend Modular: ES6 Modules & Padrão em Camadas

Objetivos

Objetivos da Atividade (Módulo 2 — ES6 Modules & Padrão em Camadas)
Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Superar a arquitetura monolítica de arquivo único (`app.js`), organizando o frontend em múltiplos módulos especializados utilizando **ES6 Modules** (`import` e `export`);
- Declarar o ponto de entrada modular na página web através da diretiva: `<script type="module" src="js/main.js"></script>`;
- Compreender e aplicar a separação estrita de responsabilidades no cliente em três camadas:
  - **Camada de Serviços (`services/`):** Regras de negócio, cálculos e persistência pura no `localStorage` sem acoplamento ao DOM;
  - **Camada de Visão (`views/`):** Renderização de cards, tabelas, modais e Toasts via *template literals* do Bootstrap 5;
  - **Camada Utilitária (`utils/`):** Funções puras reutilizáveis de formatação (moeda brasileira, telefone, máscaras);
  - **Orquestrador Central (`main.js`):** Captura de eventos do usuário e coordenação entre camadas.
- Implementar o **CRUD Completo** (*Create, Read, Update, Delete*) com persistência em armazenamento local;
- Implementar **validação visual no cliente** com feedback imediato ao usuário utilizando as classes nativas do Bootstrap 5 (`.was-validated` e `.invalid-feedback`);
- Disparar notificações flutuantes dinâmicas (**Bootstrap Toasts**) para confirmar operações de cadastro, alteração e exclusão;
- Implementar filtragem reativa em tempo real por categoria de serviço na vitrine comunitária.

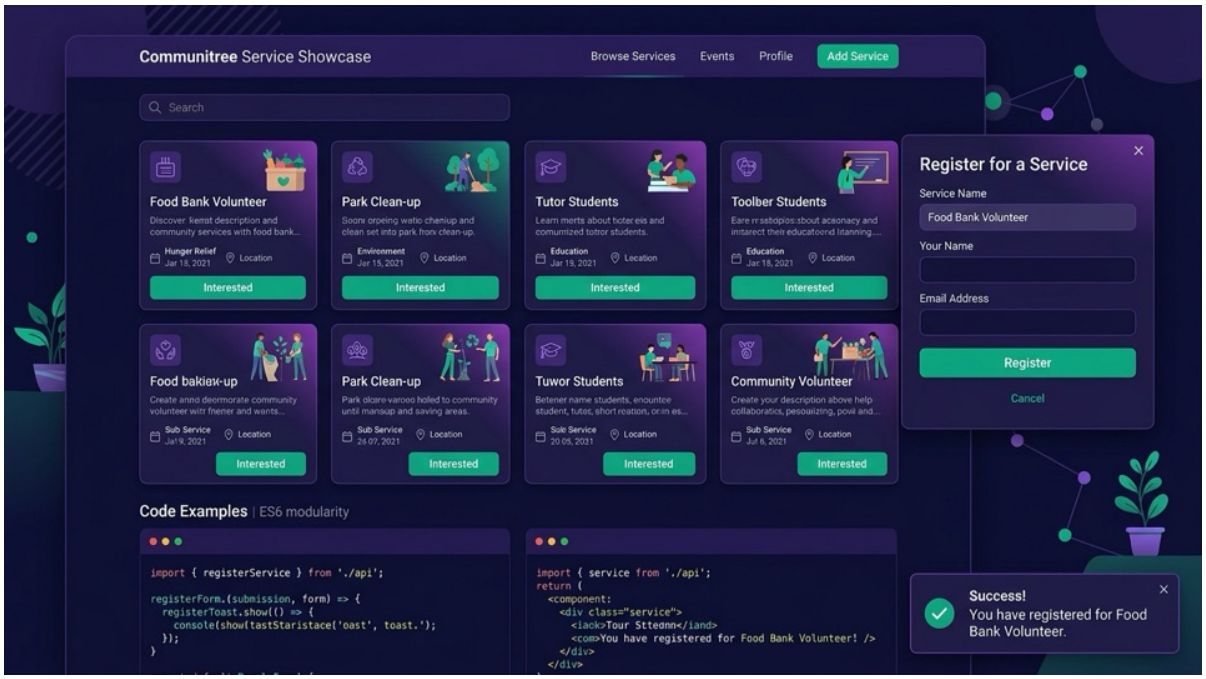


Figura 5: Visão Geral da Aplicação: Vitrine Comunitária de Garopaba com arquitetura modular em camadas ES6, modais e Toasts.

Introdução e Contextualização do Projeto

Nos Roteiros 1, 2 e 3, trabalhamos com scripts JavaScript concentrados em um único arquivo (`app.js`). Embora essa abordagem seja excelente para aprender os fundamentos, ela se torna inviável em aplicações reais de médio e grande porte, onde a mistura de seletores do DOM, regras de validação e persistência gera o chamado *"código espaguete"*.

No **Roteiro Prático 04**, damos o passo definitivo rumo à arquitetura profissional de software no frontend. Desenvolveremos a **Vitrine Comunitária de Empreendedores Locais de Garopaba**, uma aplicação que permite cadastrar, listar, filtrar, editar e gerenciar serviços da comunidade local.

Este projeto serve também como a **base tecnológica inicial para a Ação Curricularizada de Extensão na SNCT (Semana Nacional de Ciência e Tecnologia)**.

Estrutura Arquitetural de Pastas

O projeto deve ser organizado na pasta `roteiro-4/exercicio-roteiro4/` seguindo rigorosamente a estrutura modular:

Árvore de Diretórios Modular do Projeto:

```
roteiro-4/exercicio-roteiro4/
|-- index.html      <-- Vitrine e forms (BS5)
|-- styles.css      <-- Estilos e transições
`-- js/
```

```
|-- main.js    <-- Orquestrador central
|-- services/
|   `-- vitrineService.js <-- CRUD Storage
|-- views/
|   `-- vitrineView.js    <-- HTML dinâmico
`-- utils/
    `-- formatters.js     <-- Formatações
```

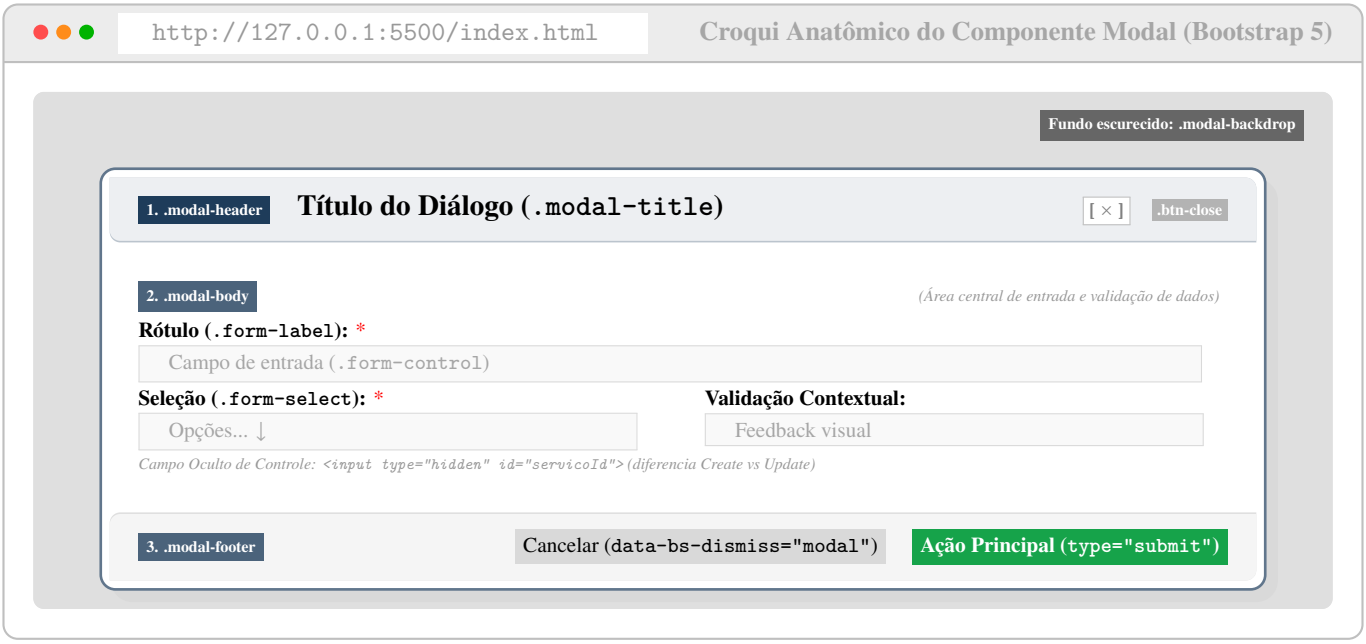
Fundamentação Teórica: Componentes Modernos de Interface (Bootstrap 5)

Antes de iniciarmos a implementação prática guiada, é fundamental compreender a finalidade, o comportamento e a anatomia dos principais componentes de interface gráfica utilizados na aplicação:

5.3.1 1. Janelas Modais (*Modals*): Diálogos Sobrepostos de Alta Atenção

Uma **janela modal** (ou diálogo modal) é um elemento flutuante que surge acima de todo o conteúdo da página, bloqueando temporariamente a interação com a tela de fundo por meio de uma camada escurecida semi-transparente chamada ***backdrop***.

- **Finalidade de Uso:** Concentrar a atenção exclusiva do usuário em uma tarefa crítica (como preencher o formulário de cadastro ou atualizar dados de um serviço) sem sair da tela atual nem recarregar o navegador (*Single Page Application*).



Anatomia no Bootstrap 5 (conforme ilustrado acima):

Elemento / Classe	Camada Anatômica	Papel Arquitetural e Comportamento
`.modal`	Contêiner Base	Invisível por padrão (`display: none`), ativa a transição animada suave (`.fade`) e o bloqueio de tela (*backdrop*).
`.modal-dialog`	Geometria & Posição	Controla largura responsiva e o alinhamento vertical no viewport via `.modal-dialog-centered`.
`.modal-content`	Invólucro do Diálogo	Superfície branca com cantos arredondados e elevação (`.shadow-lg`), estruturada em 3 seções:
`.modal-header`	1. Cabeçalho	Título da ação (`.modal-title`) e botão de fechar nativo (`.btn-close` com `data-bs-dismiss="modal"`).
`.modal-body`	2. Corpo Central	Área nobre do formulário com rótulos, campos de entrada, validações e campo oculto de controle (`#servicoId`).
`.modal-footer`	3. Rodapé de Ações	Área de decisão e submissão com botões de controle (*Cancelar* e *Salvar*).
`bootstrap.Modal`	Controle JavaScript	Instanciação programática (`getOrCreateInstance`) para abertura (`.show()`) e fechamento (`.hide()`).

5.3.2 2. Notificações Flutuantes (*Toasts*): Feedback Assíncrono Não-Intrusivo

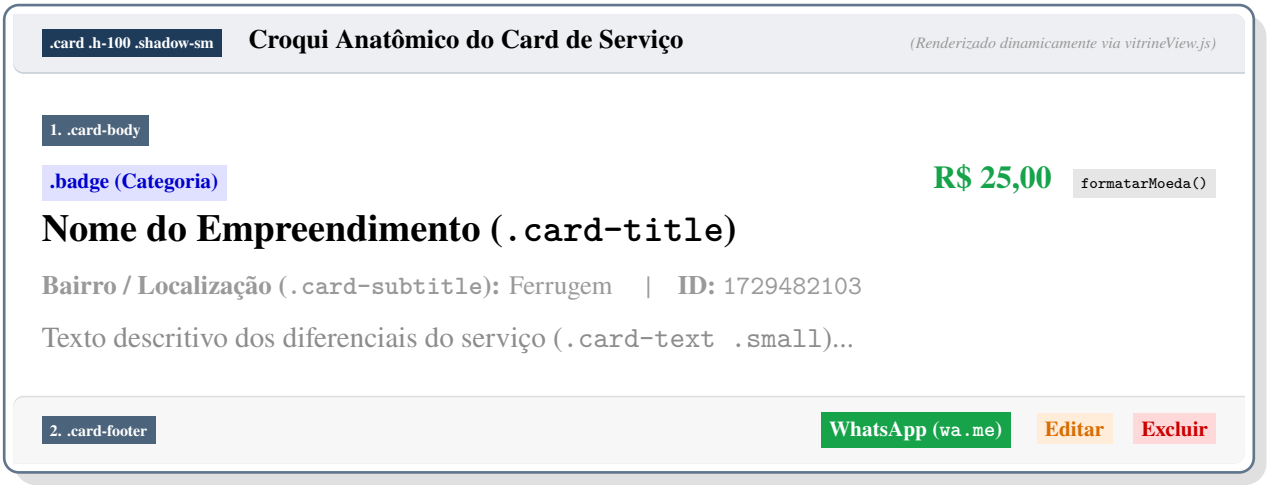
O componente **Toast** é uma notificação compacta e temporária que surge sobreposta em uma posição fixa da tela (canto superior direito) para confirmar o sucesso de operações (ex: *"Serviço cadastrado com sucesso!"*).

- **Vantagem sobre o `alert()`:** O diálogo nativo `window.alert()` é síncrono e bloqueante: ele congela a execução de qualquer outro script JavaScript no navegador e força o usuário a clicar em "OK" para continuar. O **Toast**, por outro lado, é assíncrono, suave e desaparece automaticamente após alguns segundos (*autohide*), proporcionando uma experiência de usuário profissional;

- **Posicionamento com `.toast-container`:** O elemento contêiner utiliza classes utilitárias de posicionamento fixo (`position-fixed top-0 end-0 p-3`) com índice z prioritário (`z-index: 1090`), garantindo visibilidade imediata sem deslocar os demais elementos do layout.

5.3.3 3. Cartões de Conteúdo (*Cards*): Unidades Modulares e Grid Uniforme

Um **Card** é um contêiner retangular flexível (`.card .shadow-sm`) que agrupa de forma coesa todas as informações e ações pertinentes a um único registro:



Anatomia no Bootstrap 5 (conforme ilustrado acima):

Elemento / Classe	Camada Anatômica	Papel Arquitetural e Comportamento
`.card`	Contêiner Modular	Unidade visual básica com borda suave e elevação sutil (`.shadow-sm`) que agrupa os dados de um serviço.
`.h-100`	Equalização Vertical	Força 100% da altura da coluna no grid Flexbox, garantindo nivelamento e alinhamento visual uniforme na vitrine.
`.card-body`	1. Corpo Principal	Abriga a badge de categoria, o título comercial (`.card-title`), o preço formatado, a localização e a descrição.
`.card-footer`	2. Rodapé de Ações	Fundo neutro transparente (`.bg-transparent`) agrupando os botões de ação: WhatsApp, Editar e Excluir.

5.3.4 4. Validação Visual e o Campo Oculto (`type="hidden"`)

- **Atributo `novalidate`:** Informa ao navegador para suprimir os balões de validação nativos, permitindo que a aplicação assuma o controle visual das mensagens de erro com o Bootstrap: ao adicionar a classe `.was-validated` ao formulário, cada campo é colorido conforme o resultado da validação nativa do HTML5 (verde para válido, vermelho para inválido) e os contêineres contextuais `.invalid-feedback` dos campos inválidos são exibidos;
- **Papel Arquitetural do Campo Oculto (`#servicoId`):** O campo com `type="hidden"` não é renderizado visualmente para o usuário, mas é o elemento-chave que diferencia as operações de **Criação** (*Create*) e **Atualização** (*Update*) no padrão CRUD: se o campo estiver vazio, o clique em ”Salvar” cria um novo serviço; se contiver um ID, atualiza o serviço correspondente mantendo o identificador original.

Fundamentação Teórica: Arquitetura Modular com ES6 Modules

5.4.1 1. Declaração no HTML (`type="module"`)

Ao declarar `<script type="module" src="js/main.js"></script>`, o navegador aplica comportamentos modernos essenciais:

- **Modo Estrito Automático (`"use strict"`):** Impede o uso de variáveis não declaradas e falhas silenciosas;
- **Escopo Fechado de Módulo:** Variáveis e funções criadas dentro de um arquivo não poluem o escopo global (`window`);
- **Carregamento Assíncrono com Adiamento (`defer`):** O script aguarda o parsing completo do DOM antes de executar.

5.4.2 2. Exportação e Importação (`export` e `import`)

Para compartilhar recursos entre arquivos, usamos a sintaxe explícita de módulos:

Exemplo de Exportação Nomeada e Importação:

```
// No arquivo js/utils/formatters.js
export function formatarTelefone(telefone) {
  return telefone.replace(
    /^(\d{2})(\d{5})(\d{4})/,
    '($1) $2-$3'
  );
}

// No arquivo js/main.js
import { formatarTelefone }
  from './utils/formatters.js';

// Resultado: (48) 99999-8888
console.log(formatarTelefone('48999998888'));
```

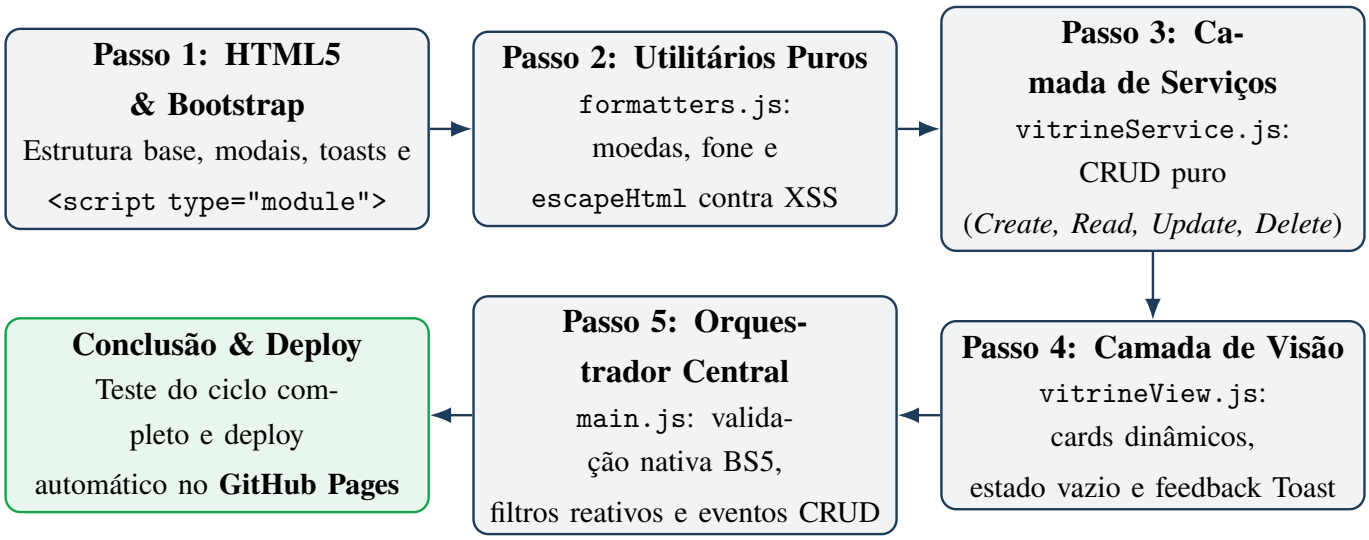


Figura 6: Pipeline arquitetural e sequência cronológica de implementação do Roteiro 04.

Links diretos para o GitHub (Código-Fonte de Referência):

Caso prefira consultar, clonar ou comparar os arquivos diretamente no repositório:

- **index.html** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-inter blob/main/roteiro4-novamente/index.html`
- **main.js (orquestrador)** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-internet-2/blob/main/roteiro4-novamente/js/main.js`
- **styles.css** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-intern blob/main/roteiro4-novamente/styles.css`
- **formatters.js** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-int blob/main/roteiro4-novamente/js/utils/formatters.js`
- **vitrineService.js** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para- blob/main/roteiro4-novamente/js/services/vitrineService.js`
- **vitrineView.js** → `https://github.com/chameoandre/CST-Sistemas-para-internet-programacao-para-in blob/main/roteiro4-novamente/js/views/vitrineView.js`

Passo a Passo Guiado do Roteiro

Antes de começar:

- **Abra sempre pelo Live Server.** Módulos ES6 não carregam quando o `index.html` é aberto com duplo clique (endereço `file://`).
- **Ao copiar código deste PDF**, copie uma página por vez. Quando um bloco continua na página seguinte, o cabeçalho e o rodapé do documento vêm junto na seleção e precisam ser apagados.
- **Código no GitHub:** Os links diretos para cada arquivo no repositório estão disponíveis no quadro da página anterior.

Fase 1.1/5: Estrutura HTML5 Base e Ponto de Entrada (index.html)

Responsabilidade Arquitetural: Estabelecer o esqueleto visual da página, carregar as dependências de estilo (Bootstrap 5 e Icons), fornecer os pontos de ancoragem da interface (`#vitrineContainer`, `#toastNotificacao`, `#totalServicosBadge` e `#filtroCategoria`) e conectar o módulo raiz `js/main.js`.




```
<!-- index.html (Parte 1: Estrutura Base e Scripts) -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vitrine Comunitária de Garopaba - Roteiro 04 (ES6 Modules)</title>
  <!-- Bootstrap 5 CSS, Icons & Google Fonts -->
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
```

```
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link rel="stylesheet"
    href="https://fonts.googleapis.com/css2?family=Outfit:wght@500;700;800&display=swap">
  <link rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700&display=swap">
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <!-- Container de Toasts com corpo #toastMensagem -->
  <div class="toast-container position-fixed top-0 end-0 p-3" style="z-index: 1090;">
    <div id="toastNotificacao"
         class="toast align-items-center text-bg-success border-0 shadow-lg"
         role="alert">
      <div class="d-flex">
        <div class="toast-body d-flex align-items-center gap-2">
          <i class="bi bi-check-circle-fill fs-5"></i>
          <div>
            <strong id="toastTitulo" class="d-block">Sucesso!</strong>
            <span id="toastMensagem">Operação realizada com êxito.</span>
          </div>
        </div>
        <button type="button" class="btn-close btn-close-white me-2 m-auto"
                data-bs-dismiss="toast"></button>
      </div>
    </div>
  </div>

  <!-- Hero Banner com Tipografia Outfit e Badges -->
  <header class="hero-banner shadow-sm">
    <div class="container">
      <div class="d-flex align-items-center justify-content-between flex-wrap gap-3">
        <div>
          <div class="d-flex align-items-center gap-2 mb-2">
            <span class="badge bg-warning text-dark font-outfit fw-bold px-3 py-2 rounded-pill">
              <i class="bi bi-code-slash me-1"></i> ROTEIRO PRÁTICO 04
            </span>
            <span class="badge bg-white text-success font-outfit fw-bold px-3 py-2 rounded-pill">
              ES6 MODULES &amp;amp; ARQUITETURA EM CAMADAS
            </span>
          </div>
          <h1 class="display-6 fw-bold font-outfit mb-1">Vitrine Comunitária de Empreendedores</h1>
          <p class="mb-0 opacity-75 small">
            Guia de serviços e comércios locais de Garopaba (SC) -
            Projeto Base de Extensão Universitária
          </p>
        </div>
        <div class="d-flex align-items-center gap-2">
          <span class="badge-counter" id="totalServicosBadge">Carregando...</span>
          <button data-bs-toggle="modal" data-bs-target="#modalCadastro"
            class="btn btn-warning text-dark font-outfit fw-bold px-3 py-2 rounded-pill shadow-sm">
            <i class="bi bi-plus-circle-fill me-1"></i> Divulgar Serviço
          </button>
        </div>
      </div>
    </div>
  </header>

  <main class="container mb-5">
    <!-- Barra de Controle e Filtros -->
    <div class="card border-0 shadow-sm rounded-4 mb-4 p-3 bg-white">
```

```
      <div class="row g-3 align-items-center justify-content-between">
        <div class="col-md-5">
          <div class="input-group">
            <span class="input-group-text bg-light border-end-0">
              <i class="bi bi-funnel-fill text-muted"></i>
            </span>
            <select class="form-select border-start-0 bg-light" id="filtroCategoria">
              <option value="todas">Todas as Categorias</option>
              <option value="Alimentação">Alimentação &amp;amp; Gastronomia</option>
              <option value="Tecnologia">Tecnologia &amp;amp; Internet</option>
              <option value="Artesanato">Artesanato &amp;amp; Arte Local</option>
              <option value="Serviços Gerais">Serviços Gerais &amp;amp; Reformas</option>
              <option value="Turismo">Turismo &amp;amp; Passeios</option>
            </select>
          </div>
        </div>

        <div class="col-md-7 text-md-end text-muted small">
          <span class="badge bg-light text-dark border me-1">
            <i class="bi bi-folder-fill text-warning me-1"></i> js/services/
          </span>
          <span class="badge bg-light text-dark border me-1">
            <i class="bi bi-layout-text-window-reverse text-info me-1"></i> js/views/
          </span>
          <span class="badge bg-light text-dark border">
            <i class="bi bi-gear-wide-connected text-success me-1"></i> js/utils/
          </span>
        </div>
      </div>
    </div>

    <!-- Ponto de Montagem dos Cards Dinâmicos -->
    <div class="row" id="vitrineContainer"></div>
  </main>

  <!-- Bootstrap Bundle JS + Entrada Modular ES6 -->
  <script
    src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  <script type="module" src="js/main.js"></script>
</body>
</html>
```

Checkpoint de Validação 1.1: Como Testar no Navegador

Abra o `index.html` no navegador via **Live Server**:

1. Título, badges, contador (*"Carregando..."*) e botão **"Divulgar Serviço"** visíveis (o degradê verde do banner só surgirá com o `styles.css` no Passo 1.3);
2. Seletor de categorias locais e badges dos módulos visíveis e funcionais;
3. Área central vazia, pronta para receber os cards dinâmicos via JavaScript.

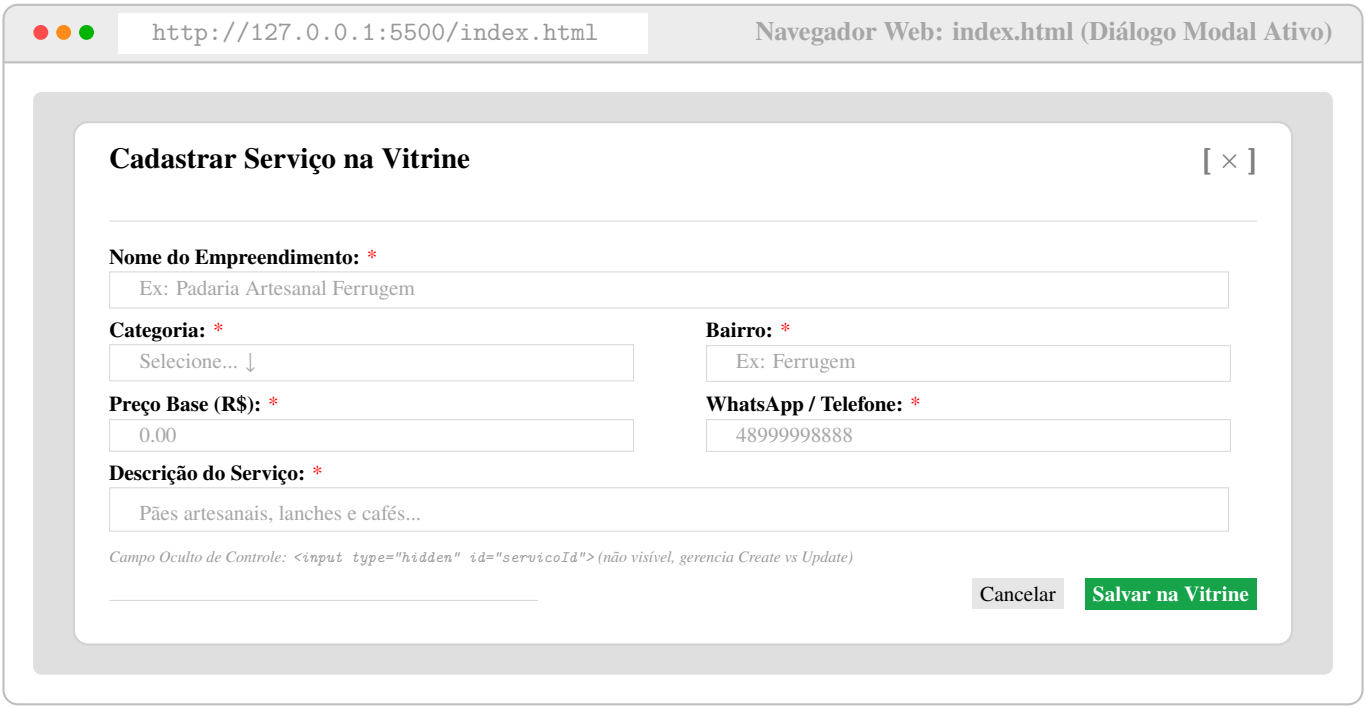
Nota: Erros 404 no Console para `styles.css` e `js/main.js` são esperados até os Passos 1.3 e 5. O modal abrirá após estruturado no Passo 1.2.

Fase 1.2/5: Formulário Modal de Cadastro e Edição (index.html)

Responsabilidade Arquitetural: Fornecer a interface de diálogo acessível para inserção e atualização de serviços, contendo validações visuais nativas do HTML5 (`required`), seletores padronizados e um campo oculto (`#servicoId`) que diferencia operações de *Create* (novo) e *Update* (edição).

Onde e como inserir este código no `index.html`:

- **Atenção: Não apague nem substitua** o código estruturado no Passo 1.1! Todo o documento construído anteriormente permanece intacto.
- **Local exato de inserção:** Insira o bloco completo do modal logo após a tag de fechamento `</main>` e imediatamente antes das tags de script (`<!-- Bootstrap Bundle JS... -->`).




```
<!-- index.html (Inserir logo após </main> e imediatamente antes de <script>) -->
<!-- ========================================================================== -->
<!-- PARTE 2: MODAL DE CADASTRO E EDIÇÃO (Não apaga a Parte 1!)                 -->
<!-- ========================================================================== -->
<div class="modal fade" id="modalCadastro" tabindex="-1" aria-hidden="true">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content rounded-4 border-0 shadow-lg">
      <div class="modal-header border-0 pb-0">
        <h5 class="modal-title font-outfit fw-bold text-dark" id="modalCadastroLabel">
          <i class="bi bi-shop text-success me-2"></i>Cadastrar Serviço na Vitrine
        </h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"
                aria-label="Fechar"></button>
      </div>
      <form id="formCadastro" novalidate>
        <input type="hidden" id="servicoId" value="">
        <div class="modal-body pt-3">
```

```
<p class="text-muted small mb-3">Preencha as informações do empreendedor local:</p>

<div class="mb-3">
  <label for="nome" class="form-label small fw-bold text-muted">
    Nome do Empreendimento *
  </label>
  <input type="text" id="nome" class="form-control rounded-3" required
         placeholder="Ex: Maré Alta Artesanatos">
  <div class="invalid-feedback">Por favor, informe o nome do empreendimento.</div>
</div>

<div class="row g-2 mb-3">
  <div class="col-md-6">
    <label for="categoria" class="form-label small fw-bold text-muted">Categoria *</label>
    <select id="categoria" class="form-select rounded-3" required>
      <option value="" selected disabled>Selecione...</option>
      <option value="Alimentação">Alimentação &amp;amp; Gastronomia</option>
      <option value="Tecnologia">Tecnologia &amp;amp; Internet</option>
      <option value="Artesanato">Artesanato &amp;amp; Arte Local</option>
      <option value="Serviços Gerais">Serviços Gerais &amp;amp; Reformas</option>
      <option value="Turismo">Turismo &amp;amp; Passeios</option>
    </select>
    <div class="invalid-feedback">Selecione uma categoria.</div>
  </div>
  <div class="col-md-6">
    <label for="bairro" class="form-label small fw-bold text-muted">
      Bairro / Localidade *
    </label>
    <input type="text" id="bairro" class="form-control rounded-3" required
           placeholder="Ex: Ferrugem, Centro...">
    <div class="invalid-feedback">Informe o bairro em Garopaba.</div>
  </div>
</div>

<div class="row g-2 mb-3">
  <div class="col-md-6">
    <label for="precoBase" class="form-label small fw-bold text-muted">
      Preço Base (R$) *
    </label>
    <input type="number" id="precoBase" class="form-control rounded-3" required
           step="0.01" min="0" placeholder="Ex: 50.00">
    <div class="invalid-feedback">Informe um valor válido em R$.</div>
  </div>
  <div class="col-md-6">
    <label for="telefone" class="form-label small fw-bold text-muted">
      WhatsApp / Telefone *
    </label>
    <input type="tel" id="telefone" class="form-control rounded-3" required
           pattern="[0-9]{10,11}" placeholder="48999998888">
    <div class="invalid-feedback">Informe com DDD (10 ou 11 dígitos).</div>
  </div>
</div>

<div class="mb-3">
  <label for="descricao" class="form-label small fw-bold text-muted">
    Descrição dos Serviços e Produtos *
  </label>
  <textarea id="descricao" class="form-control rounded-3" rows="3" required
            placeholder="Descreva os produtos e diferenciais..."></textarea>
  <div class="invalid-feedback">Insira uma breve descrição sobre o negócio.</div>
</div>
```

```
        </div>

        <div class="modal-footer border-0 pt-0">
          <button type="button" data-bs-dismiss="modal"
                  class="btn btn-light rounded-pill px-4 text-muted fw-bold">Cancelar</button>
          <button type="submit" id="btnSalvarModal"
                  class="btn btn-success rounded-pill px-4 font-outfit fw-bold shadow-sm">
            <i class="bi bi-check2-circle me-1"></i>
            <span id="btnSalvarTexto">Salvar na Vitrine</span>
          </button>
        </div>
      </form>
    </div>
  </div>
</div>
```

Checkpoint de Validação 1.2: Como Testar a Abertura do Modal

Atualize o navegador e clique no botão amarelo **"Divulgar Serviço"**:

1. O modal deve abrir de forma suave, centralizada e com bordas arredondadas (`.rounded-4`);
2. Verifique se todos os campos estão bem espaçados no corpo do modal;
3. Clique em **"Cancelar"** ou no botão **"X"** para confirmar o fechamento nativo.

Fase 1.3/5: Estilização Complementar (styles.css)

Responsabilidade Arquitetural: Definir variáveis de cores institucionais do IFSC Garopaba, aplicar tipografia Google Fonts (*Outfit* e *Plus Jakarta Sans*), o gradiente no cabeçalho e a elevação suave nos cards (`transform: translateY(-5px)`).

```
/* styles.css */
:root {
  --ifsc-green: #059669;
  --ifsc-dark-green: #047857;
  --garopaba-blue: #0284c7;
  --bg-page: #f8fafc;
  --text-dark: #0f172a;
}
body {
  font-family: 'Plus Jakarta Sans', sans-serif;
  background-color: var(--bg-page);
  color: var(--text-dark);
}
.font-outfit { font-family: 'Outfit', sans-serif; }
.hero-banner {
  background: linear-gradient(135deg, #064e3b 0%, #065f46 50%, #047857 100%);
  color: #ffffff;
  padding: 2.5rem 0 2rem;
  margin-bottom: 2rem;
  border-bottom: 3px solid #10b981;
}
.vitrine-card {
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  background: #ffffff;
```

```
}
.vitrine-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 15px 30px -10px rgba(0,0,0,0.12) !important;
}
.badge-counter {
  background: rgba(255,255,255,0.2);
  backdrop-filter: blur(5px);
  border: 1px solid rgba(255,255,255,0.3);
  padding: 0.4rem 0.9rem;
  border-radius: 50px;
  font-size: 0.85rem;
  font-weight: 700;
}
```

Checkpoint de Validação 1.3: Como Testar a Estilização

Salve `styles.css` na mesma pasta do `index.html` e atualize a página:

1. O banner superior exibe o **degradê verde** com tipografia branca;
2. O contador surge como uma pílula translúcida ao lado de **"Divulgar Serviço"**;
3. No Console (F12), o erro 404 de `styles.css` desaparece (o de `js/main.js` segue até o Passo 5).

Fase 2/5: Utilitários Puros (js/utils/formatters.js)

Responsabilidade Arquitetural: Funções puras e reutilizáveis de transformação de dados. Recebem entradas brutas e retornam formatos amigáveis para o usuário brasileiro (moeda e telefone), além de realizar a sanitização de strings contra ataques XSS (*Cross-Site Scripting*). Não tocam no DOM nem no armazenamento.

Croqui Visual 2: Fluxo Puro de Entrada e Saida

```
[ "48999998888" ] --------> [ formatarTelefone() ] --------> "(48) 99999-8888"
[ 35.5          ] --------> [ formatarMoeda()    ] --------> "R$ 35,50"
[ "<script>..." ] --------> [ escapeHtml()       ] --------> "&lt;script&gt;..."
```

```
// js/utils/formatters.js
export function formatarMoeda(valor) {
  const num = parseFloat(valor) || 0;
  return num.toLocaleString('pt-BR', {
    style: 'currency',
    currency: 'BRL'
  });
}

export function formatarTelefone(fone) {
  const digits = (fone || '').replace(/\D/g, '');
  if (digits.length === 11) {
    return digits.replace(
      /^(\d{2})(\d{5})(\d{4})/,
      '($1) $2-$3'
    );
  }
  return fone;
}

export function escapeHtml(texto) {
  const div = document.createElement('div');
  div.textContent = texto;
  return div.innerHTML;
}
```

Checkpoint de Validação 2: Teste via Console do Navegador (F12)

No Console do navegador (**F12**), execute:

```
import('./js/utils/formatters.js').then(m => console.log(m.formatarMoeda(149.9)));
```

Resultado esperado: `"R$ 149,90"`. Comprova a exportação nomeada e a formatação nativa!

Fase 3/5: Camada de Serviços e Dados (js/services/vitrineService.js)

Responsabilidade Arquitetural: Isolar completamente a regra de persistência e ciclo de vida do CRUD (*Create, Read, Update, Delete*). Opera estritamente sobre a chave `'garopaba_vitrine_servicos'` do `localStorage` do navegador, sem gerar nenhuma tag HTML nem manipular o DOM.

Croqui Visual 3: Arquitetura da Camada de Servicos e Carga Inicial

```
[ DADOS_INICIAIS ] ---> [ carregarDadosIniciais() ] ---> [ LocalStorage ]
                                                                |
[ Interface / main.js ] <--(obterServicos, salvar, atualizar)---+
```

```javascript
// js/services/vitrineService.js
const STORAGE_KEY = 'garopaba_vitrine_servicos';

// Dados modelo semente para enriquecer a experiência inicial
export const DADOS_INICIAIS = [
  {
    id: '1',
    nome: 'Maré Alta Artesanatos & Cerâmicas',
    categoria: 'Artesanato',
    bairro: 'Centro Histórico',
    precoBase: 35.00,
    telefone: '48991234567',
    descricao: 'Peças artesanais e utilitárias modeladas à mão ' +
      'com argila local e conchas de Garopaba.'
  },
  {
    id: '2',
    nome: 'Garopaba Web & Design Studio',
    categoria: 'Tecnologia',
    bairro: 'Ferrugem',
    precoBase: 150.00,
    telefone: '48998765432',
    descricao: 'Criação de websites profissionais responsivos, ' +
      'cardápios digitais e suporte para comércio local.'
  },
  {
    id: '3',
    nome: 'Pescado Fresco do Zequinha',
    categoria: 'Alimentação',
    bairro: 'Canto das Canoas',
    precoBase: 42.00,
    telefone: '48984561234',
    descricao: 'Peixes frescos e frutos do mar da pesca artesanal ' +
      'diária entregues com higiene e pontualidade.'
  }
];

// Função dedicada para carregar os dados modelo no LocalStorage
export function carregarDadosIniciais() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(DADOS_INICIAIS));
  return DADOS_INICIAIS;
```

```
}

// READ: Recupera os serviços ou carrega os dados modelo se vazio
export function obterServicos() {
  const dados = localStorage.getItem(STORAGE_KEY);
  if (!dados) {
    return carregarDadosIniciais();
  }
  try {
    return JSON.parse(dados);
  } catch (e) {
    console.error('Erro ao processar dados da vitrine:', e);
    return [];
  }
}

export function salvarServico(novo) {
  const servicos = obterServicos();
  const completo = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novo
  };
  servicos.unshift(completo);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return completo;
}

export function atualizarServico(id, dados) {
  const servicos = obterServicos();
  const idx = servicos.findIndex(s => s.id === id);
  if (idx !== -1) {
    servicos[idx] = { ...servicos[idx], ...dados,
                      dataEdicao: new Date().toISOString() };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return servicos[idx];
  }
  return null;
}

export function removerServico(id) {
  const servicos = obterServicos().filter(s => s.id !== id);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicos;
}
```

Checkpoint de Validação 3: Teste de Persistência no Console (F12)

No Console (**F12**), teste a carga e a persistência dos dados:

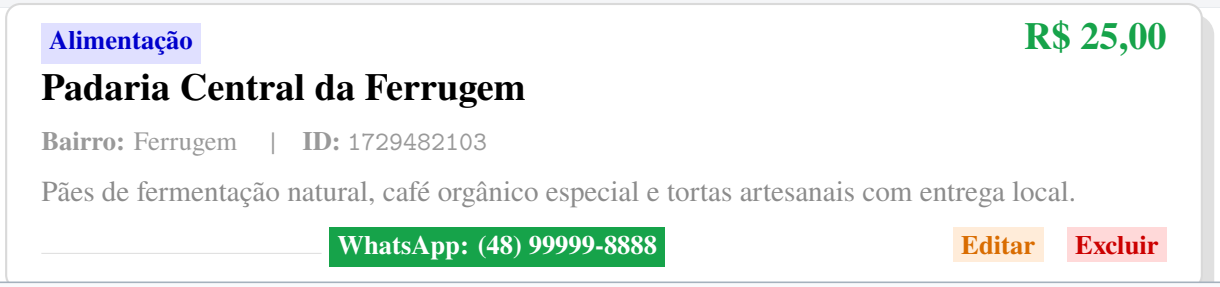
```
import('./js/services/vitrineService.js').then(s => {
  s.carregarDadosIniciais();
  console.log('Dados salvos:', s.obterServicos());
});
```

Resultado esperado: Array com os 3 serviços salvo em `garopaba_vitrine_servicos` (verifique

também na aba **Application** → **Local Storage**).

Fase 4/5: Camada de Visão (js/views/vitrineView.js)

Responsabilidade Arquitetural: Construir e manipular os elementos visuais na interface. Transforma objetos de dados em cards HTML (`criarCardHtml`), renderiza coleções no contêiner (`renderizarCards`) e dispara notificações flutuantes (`exibirToast`).




```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml }
  from '../utils/formatters.js';

// Mapeamento de cores para categorias
const CORES_CATEGORIA = {
  'Alimentação': 'success',
  'Tecnologia': 'info',
  'Artesanato': 'warning',
  'Serviços Gerais': 'primary',
  'Turismo': 'secondary'
};

// Função para criar o HTML de um card de serviço
export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');
  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
              <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>
          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>
          <hr class="my-3 text-muted opacity-25">
          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank"
               class="btn btn-sm btn-outline-success rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
```

```
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-primary btn-editar rounded-pill px-2"
                      data-id="${s.id}" title="Editar serviço">
                <i class="bi bi-pencil-square"></i>
              </button>
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2"
                      data-id="${s.id}" title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>`;
}

// Renderização dinâmica dos cards e tratamento de estado vazio
export function renderizarCards(servicos, containerElement) {
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 text-center py-5">
        <div class="p-4 rounded-4 bg-light border">
          <i class="bi bi-inbox fs-1 text-muted d-block mb-2"></i>
          <h5 class="fw-bold text-secondary mb-1">Nenhum serviço cadastrado nesta categoria</h5>
          <p class="text-muted small mb-0">Cadastre um novo serviço ou altere o filtro acima.</p>
        </div>
      </div>`;
    return;
  }
  containerElement.innerHTML = servicos.map(criarCardHtml).join('');
}

// Notificações flutuantes assíncronas (Toasts) com auto-fechamento
export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastMsg = document.getElementById('toastMensagem');
  const toastTitulo = document.getElementById('toastTitulo');

  if (toastEl && toastMsg) {
    toastMsg.innerText = mensagem;
    if (toastTitulo) {
      toastTitulo.innerText = tipo === 'success' ? 'Sucesso!' : 'Aviso';
    }
    toastEl.className = `toast align-items-center text-bg-${tipo} border-0 shadow-lg`;
    const toast = bootstrap.Toast.getOrCreateInstance(toastEl, { delay: 4000 });
    toast.show();
  }
}
```

Checkpoint de Validação 4: Teste do Toast no Console (F12)

No Console (**F12**), teste a notificação flutuante:

```
import('./js/views/vitrineView.js').then(v => v.exibirToast('Notificação funcionando!'));
```

Resultado esperado: O Toast verde surge no canto superior direito da tela.

Fase 5/5: Orquestrador Central (js/main.js)

Responsabilidade Arquitetural: O controlador principal da aplicação. Importa os serviços e as visões, escuta o carregamento do DOM, conecta os eventos de submissão do formulário (*Create & Update*), a delegação de cliques nos cards (*Edit & Delete*) e a reatividade instantânea dos filtros.

Croqui Visual 5: Ciclo de Vida da Orquestracao

```
Usuario clica/submete ---> main.js valida ---> vitrineService (localStorage)
                                                        |
Interface renderiza cards <--- vitrineView <----------+ (atualizarInterface)
```

```
// js/main.js
import {
  obterServicos, salvarServico,
  atualizarServico, removerServico
} from './services/vitrineService.js';
import {
  renderizarCards, exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const modalEl = document.getElementById('modalCadastro');
  const servicoIdInput = document.getElementById('servicoId');

  function atualizarInterface() {
    const todos = obterServicos();
    const filtro = filtroCategoria.value;
    const filtrados = filtro === 'todas'
      ? todos
      : todos.filter(s => s.categoria === filtro);
    renderizarCards(filtrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${todos.length} Serviços Cadastrados`;
    }
  }

  // Filtros Reativos
  filtroCategoria.addEventListener('change', atualizarInterface);
  atualizarInterface();

  // Reset do Modal ao abrir para novo cadastro
  if (modalEl) {
    modalEl.addEventListener('show.bs.modal', () => {
      if (!servicoIdInput.value) {
        form.reset();
        form.classList.remove('was-validated');
        const modalTitulo = document.getElementById('modalCadastroLabel');
        if (modalTitulo) {
          modalTitulo.innerHTML =
```

```
            '<i class="bi bi-shop text-success me-2"></i>Cadastrar Serviço na Vitrine';
        }
        const btnSalvarTexto = document.getElementById('btnSalvarTexto');
        if (btnSalvarTexto) {
          btnSalvarTexto.innerText = 'Salvar na Vitrine';
        }
      }
    });

    modalEl.addEventListener('hidden.bs.modal', () => {
      form.reset();
      servicoIdInput.value = '';
      form.classList.remove('was-validated');
    });
  }

  // Submissão do Formulário (Create & Update)
  form.addEventListener('submit', (e) => {
    e.preventDefault();
    if (!form.checkValidity()) {
      e.stopPropagation();
      form.classList.add('was-validated');
      return;
    }

    const idAtual = servicoIdInput ? servicoIdInput.value : '';
    const getVal = (id) => document.getElementById(id).value.trim();

    const dados = {
      nome: getVal('nome'),
      categoria: document.getElementById('categoria').value,
      bairro: getVal('bairro'),
      precoBase: parseFloat(getVal('precoBase')) || 0,
      telefone: getVal('telefone'),
      descricao: getVal('descricao')
    };

    if (idAtual) {
      atualizarServico(idAtual, dados);
      exibirToast('Serviço atualizado com sucesso!');
    } else {
      salvarServico(dados);
      exibirToast('Empreendimento cadastrado com sucesso!');
    }

    form.reset();
    if (servicoIdInput) servicoIdInput.value = '';
    form.classList.remove('was-validated');

    if (modalEl) {
      const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
      modal.hide();
    }
    atualizarInterface();
  });

  // Delegação de Cliques nos Cards (Edit & Delete)
  container.addEventListener('click', (e) => {
    const btnEditar = e.target.closest('.btn-editar');
    if (btnEditar) {
      const id = btnEditar.getAttribute('data-id');
```

```
      const item = obterServicos().find(s => s.id === id);
      if (item) {
        if (servicoIdInput) servicoIdInput.value = item.id;
        document.getElementById('nome').value = item.nome;
        document.getElementById('categoria').value = item.categoria;
        document.getElementById('bairro').value = item.bairro;
        document.getElementById('precoBase').value = item.precoBase;
        document.getElementById('telefone').value = item.telefone;
        document.getElementById('descricao').value = item.descricao;

        const modalTitulo = document.getElementById('modalCadastroLabel');
        if (modalTitulo) {
          modalTitulo.innerHTML =
            '<i class="bi bi-pencil-square text-primary me-2"></i>Editar Serviço na Vitrine';
        }
        const btnSalvarTexto = document.getElementById('btnSalvarTexto');
        if (btnSalvarTexto) {
          btnSalvarTexto.innerText = 'Salvar Alterações';
        }

        if (modalEl) {
          const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
          modal.show();
        }
      }
      return;
    }

    const btnExcluir = e.target.closest('.btn-excluir');
    if (btnExcluir) {
      const id = btnExcluir.getAttribute('data-id');
      if (confirm('Deseja realmente remover este serviço da vitrine?')) {
        removerServico(id);
        atualizarInterface();
        exibirToast('Serviço removido da vitrine.', 'warning');
      }
    }
  });
});
```

Checkpoint de Validação 5: Teste Completo da Aplicação Ponta a Ponta

Aplicação 100% integrada! Execute o fluxo completo no navegador:

1. **Validação:** Clique em ”Divulgar” e salve vazio. Os campos ficam vermelhos e nada é salvo;
2. **Create:** Preencha e salve. O card surge na vitrine e o contador soma +1 (sob o Toast);
3. **Filter:** Mude a categoria no select. Apenas serviços da categoria escolhida permanecem;
4. **Update:** Clique no lápis em um card, altere os dados no modal e salve;
5. **Delete:** Clique na lixeira, confirme o aviso e veja o item ser removido imediatamente.

Checklist de Conclusão e Critérios Avaliativos

Antes de realizar o commit final no repositório, certifique-se de validar todos os itens:

- ☐ O projeto utiliza estritamente o padrão ES6 Modules com a tag: `<script type="module" src="js/main.js"></script>`;
- ☐ A estrutura em pastas (`js/services/`, `js/views/`, `js/utils/`) foi rigorosamente respeitada;
- ☐ O arquivo `vitrineService.js` opera puramente sobre o `localStorage` sem manipular elementos do DOM;
- ☐ As quatro operações do CRUD (**Create, Read, Update e Delete**) foram implementadas e testadas com persistência no navegador;
- ☐ O formulário exibe validações visuais nativas do Bootstrap 5 (`.was-validated` e `.invalid-feedback`) quando campos obrigatórios não são preenchidos;
- ☐ A vitrine reage instantaneamente aos filtros de categoria sem recarregar a página;
- ☐ O código está devidamente publicado e em pleno funcionamento no **GitHub Pages**.

6 Roteiro Prático 05 — Assincronismo, Fetch API & Backend RESTful em Flask (SQLite)

Objetivos

Objetivos de Aprendizagem (Módulo 3 — Assincronismo & Integração de Serviços Web)
Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Compreender a transição do armazenamento isolado no navegador (`localStorage`) para a **arquitetura Cliente-Servidor real**, onde os dados são centralizados e compartilhados em rede;
- Configurar e executar um microbackend RESTful em **Python Flask**, compreendendo o papel dos protocolos HTTP, rotas, verbos (`GET`, `POST`, `PUT`, `DELETE`) e códigos de status (`200`, `201`, `400`, `404`, `500`);
- Integrar um banco de dados relacional embutido (**SQLite 3**) via arquivo persistente (`vitrine.db`), eliminando a necessidade de instalar servidores externos de banco;
- Compreender o mecanismo de segurança **CORS** (*Cross-Origin Resource Sharing*) e por que ele bloqueia requisições entre portas distintas (ex: frontend na porta `5500` e API na porta `5000`);
- Refatorar a camada de serviços (`vitrineService.js`) para operar com a moderna **Fetch API** do JavaScript nativo utilizando a sintaxe `async/await` e tratamento robusto de exceções com blocos `try/catch`;
- Implementar o padrão profissional de **Contingência Resiliente (Fallback)**: se a API do Flask estiver ativa (no laboratório), o sistema opera conectado ao banco relacional; se a página for executada no GitHub Pages ou offline, a aplicação chaveia automaticamente para o `localStorage` sem quebrar;
- Implementar estados visuais reativos no frontend com **Spinners de Carregamento** e **Badges dinâmicos de conectividade** com a API.

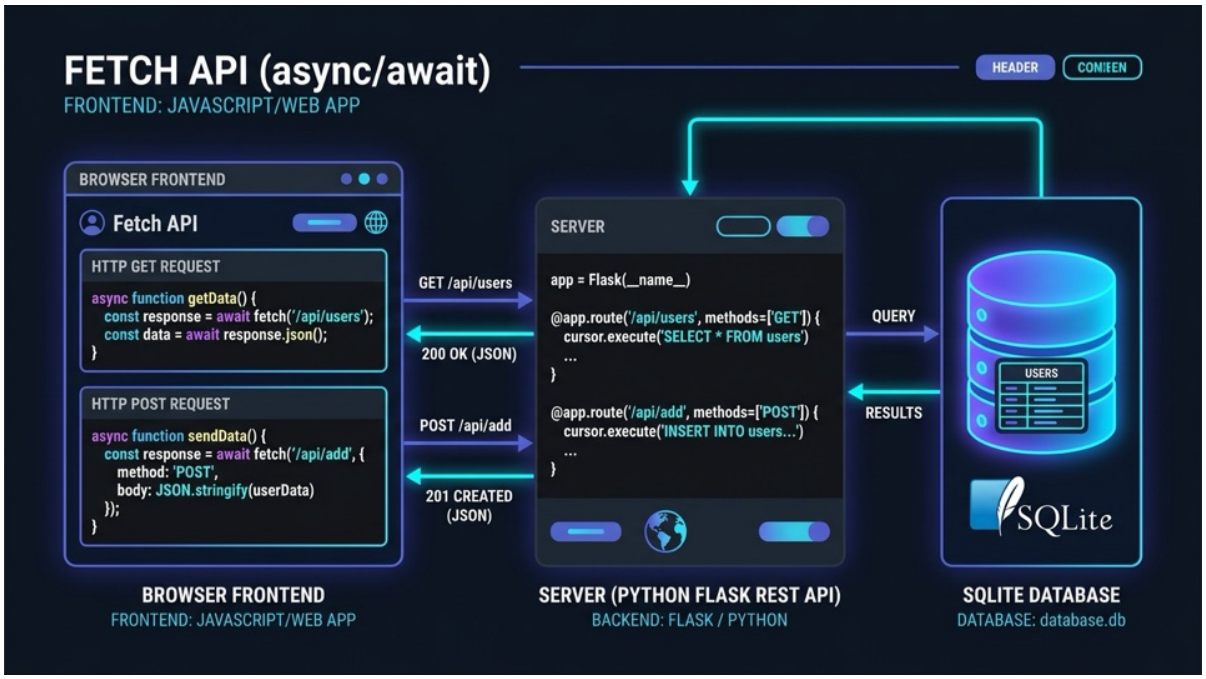


Figura 7: Visão Geral da Aplicação: Vitrine Comunitária conectada via Fetch API com backend RESTful em Flask e persistência relacional SQLite.

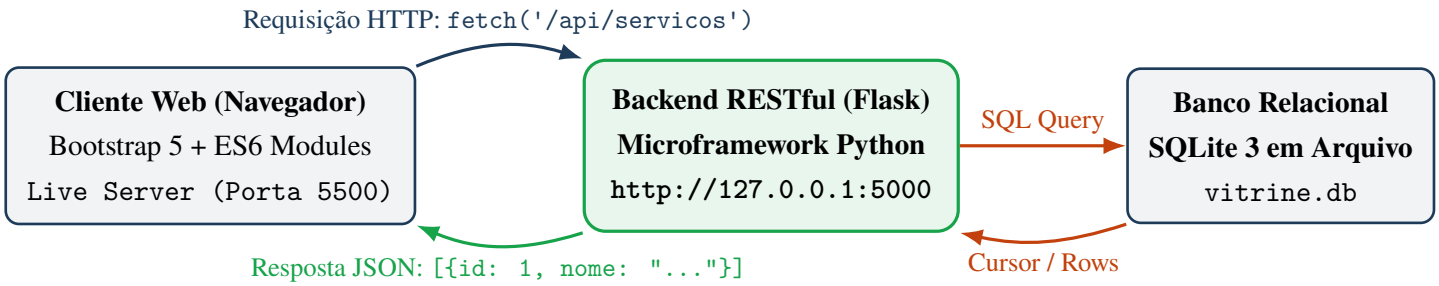
Introdução: Da Vitrine Isolada à Aplicação em Rede

No **Roteiro Prático 04**, demos um grande salto organizando o frontend na arquitetura modular com ES6 Modules (`services/`, `views/`, `utils/`). No entanto, todo o armazenamento ocorria exclusivamente dentro do navegador de cada máquina através da `Web Storage API` (`localStorage`).

Na prática profissional, se um empreendedor em Garopaba cadastrar um novo serviço em seu celular, esse serviço precisa ficar imediatamente disponível para qualquer cliente que abrir a página em outro computador. Isso exige que os dados saiam do navegador e sejam gravados em um **servidor central com banco de dados**.

Neste **Roteiro Prático 05**, evoluiremos a mesma base de código da **Vitrine Comunitária**, conectando o frontend modular a uma API RESTful leve construída em **Flask** com banco de dados relacional **SQLite**.

Visão Geral da Arquitetura Cliente-Servidor



Estrutura de Pastas do Projeto Integrado

Para manter o desacoplamento saudável entre a camada de backend e a camada de frontend, organizaremos o projeto na pasta `roteiro-5/exercicio-roteiro5/` com a seguinte estrutura:

Árvore de Diretórios do Projeto Integrado (Full-Stack Didático):

```
roteiro-5/exercicio-roteiro5/
|-- backend/
|   |-- requirements.txt   <-- Dependências Python (Flask, flask-cors)
|   |-- app.py             <-- API RESTful com rotas e conexão SQLite
|   `-- vitrine.db         <-- Banco SQLite (gerado automaticamente no 1º run)
`-- frontend/
    |-- index.html         <-- Interface com status de conexão e spinner
    |-- styles.css         <-- Estilos complementares
    `-- js/
        |-- main.js        <-- Orquestrador assíncrono de eventos
        |-- services/
        |   `-- vitrineService.js <-- Consumo Fetch API com fallback
        |-- views/
        |   `-- vitrineView.js    <-- Renderizador de cards, spinner e toasts
        `-- utils/
            `-- formatters.js     <-- Formatações (100% inalterado!)
```

Princípio do Desacoplamento Arquitetural

Observe atentamente que a camada utilitária (`formatters.js`) e grande parte da camada de visão (`vitrineView.js`) **permanecem exatamente as mesmas do Roteiro 04**. Toda a migração de tecnologia ocorre de forma isolada no `vitrineService.js`. Este é o maior benefício do padrão arquitetural que aprendemos!

Conceitos Fundamentais em Pílulas Práticas de Aquecimento

Antes de iniciarmos a construção da arquitetura completa da Vitrine Comunitária (que integrará banco de dados relacional persistente, camadas modulares em ES6 e contingência offline), realizaremos **4 mini-práticas guiadas de aquecimento**.

Cada pílula introduz um conceito arquitetural isolado e fornece o código **100% completo** para ser executado imediatamente pelos estudantes no laboratório em poucos minutos.

6.4.1 Pílula 1: Assincronismo e Leitura de Dados com Fetch API (GET)

Fundamentação Teórica: No JavaScript tradicional, operações de rede não podem ser executadas de forma síncrona (*bloqueante*), pois o navegador congelaria a interface enquanto aguardasse a resposta de outro computador.

Por isso, o JavaScript moderno utiliza o mecanismo de **Promises** (promessas de valores futuros) manipuladas pelas palavras-chave `async` e `await`. A função global `fetch(url)` dispara uma requisição HTTP

assíncrona. Quando a resposta do servidor chega, utilizamos o método `await resposta.json()` para converter o fluxo de texto HTTP em um objeto ou vetor JavaScript utilizável.

Mini-Exemplo 1: Leitura Assíncrona de Arquivo JSON com Fetch GET

Objetivo Prático: Ler um vetor de serviços em formato JSON e preencher dinamicamente uma lista HTML na tela sem recarregar a página.

Passo 1.1: Criar o arquivo de dados (`mini-exemplos/01-fetch-get/dados.json`):

```
[
  { "id": 1, "nome": "Maré Alta Artesanatos", "preco": 35.00 },
  { "id": 2, "nome": "Garopaba Web Studio", "preco": 150.00 },
  { "id": 3, "nome": "Pescado Fresco do Zequinha", "preco": 42.00 }
]
```

Passo 1.2: Criar a página de teste (`mini-exemplos/01-fetch-get/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 1: Fetch API GET</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 1: Fetch API (GET)</h5>
        <p class="text-muted small">Carregamento assíncrono de dados com async/await.</p>

        <button id="btnCarregar" class="btn btn-primary w-100 mb-3">
          Carregar Serviços Locais
        </button>

        <ul id="listaServicos" class="list-group">
          <!-- Os itens serão inseridos aqui pelo JavaScript -->
        </ul>
      </div>
    </div>
  </div>

  <script>
    const btnCarregar = document.getElementById('btnCarregar');
    const listaServicos = document.getElementById('listaServicos');

    btnCarregar.addEventListener('click', async () => {
      try {
        const resposta = await fetch('dados.json');
        const servicos = await resposta.json();

        listaServicos.innerHTML = '';
```

```
        servicos.forEach(servico => {
          const item = document.createElement('li');
          item.className = 'list-group-item d-flex justify-content-between align-items-center';
          item.innerHTML = `
            <span>${servico.nome}</span>
            <span class="badge bg-success rounded-pill">R$ ${servico.preco.toFixed(2)}</span>
          `;
          listaServicos.appendChild(item);
        });
      } catch (erro) {
        console.error('Erro ao buscar dados:', erro);
      }
    });
  </script>
</body>
</html>
```

Como testar a Pílula 1

Abra o arquivo `index.html` com a extensão **Live Server** do VS Code. Clique no botão **"Carregar Serviços Locais"** e observe os 3 itens serem renderizados instantaneamente no DOM.

6.4.2 Pílula 2: Microbackend Flask e a Política de Mesma Origem (CORS)

Fundamentação Teórica: Em uma arquitetura web real, os dados não residem em arquivos estáticos locais no navegador, mas sim em um servidor backend. No Python, o microframework **Flask** permite instanciar um servidor HTTP e declarar rotas RESTful com o decorador `@app.route()`.

Entretanto, por medidas de segurança essenciais, os navegadores implementam a **Política de Mesma Origem** (*Same-Origin Policy - SOP*). Como a nossa página frontend roda na porta `5500` (Live Server) e o Flask escuta na porta `5000`, o navegador bloqueia a requisição a menos que o backend declare explicitamente a liberação de **CORS** (*Cross-Origin Resource Sharing*) via cabeçalho `Access-Control-Allow-Origin: *`. No Flask, utilizamos a biblioteca **`flask-cors`**.

Mini-Exemplo 2: Microservidor Flask & Consumo Cross-Origin

Objetivo Prático: Executar um microservidor Python em porta distinta e consumi-lo via Fetch a partir do navegador, compreendendo a atuação do CORS.

Passo 2.1: Criar o microservidor Python (`mini-exemplos/02-micro-flask-cors/server.py`):

```python
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
# Habilita o CORS para permitir requisições de outras origens (Live Server na porta 5500)
CORS(app)

@app.route('/api/status', methods=['GET'])
def obter_status():
    return jsonify({
        "status": "online",
        "servidor": "Micro API Flask",
        "cidade": "Garopaba - SC"
    })

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Passo 2.2: Criar a página consumidora (`mini-exemplos/02-micro-flask-cors/index.html`):

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 2: Flask API & CORS</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
```

```
        <h5 class="card-title mb-3">Mini-Exemplo 2: Flask API & CORS</h5>
        <p class="text-muted small">Consumindo um microbackend Python Flask na porta 5000 a partir da porta
    5500.</p>

        <button id="btnConectar" class="btn btn-dark w-100 mb-3">
          Verificar Status da API
        </button>

        <div id="resultadoStatus" class="alert alert-secondary mb-0">
          Aguardando conexão com a API...
        </div>
      </div>
    </div>
  </div>

  <script>
    const btnConectar = document.getElementById('btnConectar');
    const resultadoStatus = document.getElementById('resultadoStatus');

    btnConectar.addEventListener('click', async () => {
      try {
        const resposta = await fetch('http://127.0.0.1:5000/api/status');
        const dados = await resposta.json();

        resultadoStatus.className = 'alert alert-success mb-0';
        resultadoStatus.innerHTML = `
          <strong>Status:</strong> ${dados.status} <br>
          <strong>Servidor:</strong> ${dados.servidor} <br>
          <strong>Origem:</strong> ${dados.cidade}
        `;
      } catch (erro) {
        resultadoStatus.className = 'alert alert-danger mb-0';
        resultadoStatus.innerHTML = `
          <strong>Erro de Conexão:</strong> Não foi possível acessar a API.<br>
          <small>Verifique se o server.py está rodando ou se houve bloqueio de CORS.</small>
        `;
      }
    });
  </script>
</body>
</html>
```

Como testar a Pílula 2 e observar o CORS em ação

1. No terminal do VS Code, execute: `python server.py`.
2. Abra o `index.html` no Live Server e clique em **"Verificar Status da API"**. O card exibirá a confirmação verde da cidade de Garopaba.
3. **Desafio Didático em Aula:** No `server.py`, comente temporariamente a linha `# CORS(app)` e reinicie o servidor. Ao clicar no botão, abra a aba *Console* do DevTools (F12) e observe o bloqueio em vermelho por violação de CORS! Em seguida, descomente a linha.

6.4.3 Pílula 3: Envio de Dados via POST (JSON.stringify e Headers HTTP)

Fundamentação Teórica: Enquanto requisições `GET` apenas solicitam dados sem alterar o estado do servidor, o verbo `POST` envia um payload de dados no corpo (*Body*) da mensagem HTTP para criação de novos registros (*Create*).

Ao utilizar a Fetch API para enviar dados com `POST`, três parâmetros de configuração são **rigorosamente obrigatórios**:

1. `method: 'POST'`: Especifica o verbo HTTP de envio;
2. `headers: { 'Content-Type': 'application/json' }`: Informa ao servidor que o corpo da mensagem trafega estruturado em formato JSON;
3. `body: JSON.stringify(objeto)`: Converte o objeto JavaScript em uma cadeia de caracteres de texto compatível com o protocolo HTTP.

No servidor Flask, o payload é recebido e decodificado automaticamente com o método `request.get_json()`.

Mini-Exemplo 3: Cadastro de Serviço via Fetch POST

Objetivo Prático: Enviar dados de um formulário via POST em formato JSON para o Flask e receber confirmação com status HTTP 201 Created.

Passo 3.1: Criar o servidor com rota POST (`mini-exemplos/03-fetch-post/server.py`):

```python
from flask import Flask, jsonify, request
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

servicos_cadastrados = []

@app.route('/api/cadastrar', methods=['POST'])
def cadastrar_servico():
    dados = request.get_json()
    if not dados or 'nome' not in dados or 'preco' not in dados:
        return jsonify({"erro": "Campos obrigatórios ausentes."}), 400

    novo_id = len(servicos_cadastrados) + 1
    registro = {
        "id": novo_id,
        "nome": dados['nome'],
        "preco": float(dados['preco'])
    }
    servicos_cadastrados.append(registro)
    print(f"[API FLASK] Novo servico cadastrado: {registro}")

    return jsonify({
        "mensagem": "Servico cadastrado com sucesso no servidor!",
        "registro": registro,
        "total": len(servicos_cadastrados)
    }), 201
```

```
if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Passo 3.2: Criar o formulário frontend (`mini-exemplos/03-fetch-post/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 3: Fetch API POST</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 3: Fetch API (POST)</h5>
        <p class="text-muted small">Enviando dados estruturados em JSON via requisição POST.</p>

        <form id="formCadastro">
          <div class="mb-3">
            <label for="campoNome" class="form-label">Nome do Serviço:</label>
            <input type="text" id="campoNome" class="form-control" placeholder="Ex: Conserto de Pranchas"
    required>
          </div>

          <div class="mb-3">
            <label for="campoPreco" class="form-label">Preço Base (R$):</label>
            <input type="number" id="campoPreco" class="form-control" step="0.01" placeholder="Ex: 80.00"
    required>
          </div>

          <button type="submit" class="btn btn-success w-100 mb-3">
            Enviar via POST para a API
          </button>
        </form>

        <div id="feedbackPost" class="alert alert-secondary d-none mb-0"></div>
      </div>
    </div>
  </div>

  <script>
    const formCadastro = document.getElementById('formCadastro');
    const campoNome = document.getElementById('campoNome');
    const campoPreco = document.getElementById('campoPreco');
    const feedbackPost = document.getElementById('feedbackPost');

    formCadastro.addEventListener('submit', async (evento) => {
      evento.preventDefault();

      const dadosParaEnvio = {
        nome: campoNome.value.trim(),
        preco: parseFloat(campoPreco.value)
      };
```

```
      try {
        const resposta = await fetch('http://127.0.0.1:5000/api/cadastrar', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json'
          },
          body: JSON.stringify(dadosParaEnvio)
        });

        if (!resposta.ok) {
          throw new Error(`Erro na resposta do servidor: código ${resposta.status}`);
        }

        const dadosResposta = await resposta.json();

        feedbackPost.className = 'alert alert-success mb-0';
        feedbackPost.classList.remove('d-none');
        feedbackPost.innerHTML = `
          <strong>${dadosResposta.mensagem}</strong><br>
          <small>ID gerado: #${dadosResposta.registro.id} | Total no servidor: ${dadosResposta.total}</small>
        `;

        formCadastro.reset();
      } catch (erro) {
        feedbackPost.className = 'alert alert-danger mb-0';
        feedbackPost.classList.remove('d-none');
        feedbackPost.innerHTML = `<strong>Falha ao enviar:</strong> ${erro.message}`;
      }
    });
  </script>
</body>
</html>
```

Como testar a Pílula 3

Com o `server.py` em execução, digite um serviço e preço no formulário e clique em **"Enviar via POST"**. Observe que o terminal do Python exibirá o log do registro recebido e a página exibirá o alerta verde com o ID gerado pelo servidor!

6.4.4 Pílula 4: Resiliência de Rede e Feedback Visual (try/catch/finally com Spinner)

Fundamentação Teórica: Em aplicações de produção, falhas de conectividade (servidor fora do ar, oscilação de Wi-Fi, erro 500) são inevitáveis. Uma aplicação de qualidade profissional precisa:

- Interceptar erros de rede com blocos `try...catch`;
- Validar o status HTTP com a propriedade booleana `resposta.ok` (que retorna `true` apenas para códigos HTTP 200–299);
- Fornecer feedback visual instantâneo durante a espera (**Spinner de Carregamento** do Bootstrap);
- Utilizar obrigatoriamente o bloco `finally`, que é executado **sempre** (tanto em caso de sucesso quanto de falha), garantindo que o spinner seja ocultado e o botão reativado.

Mini-Exemplo 4: Controle de Loading State e Tratamento de Quedas de Rede

Objetivo Prático: Implementar um botão com spinner de carregamento e tratar quedas de conectividade com feedback amigável ao usuário.

Passo 4.1: Criar a interface resiliente (`mini-exemplos/04-tratamento-erros-spinner/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 4: Tratamento de Falhas & Spinners</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="
    stylesheet">
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 520px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 4: Resiliência & Spinner</h5>
        <p class="text-muted small">
          Controle de estados de carregamento (Loading State) e tratamento resiliente de falhas de rede com
    try...catch...finally.
        </p>

        <button id="btnVerificar" class="btn btn-primary w-100 d-flex justify-content-center align-items-
    center gap-2 mb-3">
          <span id="iconeBotao"><i class="bi bi-arrow-repeat"></i></span>
          <span id="spinnerCarregamento" class="spinner-border spinner-border-sm d-none" role="status"></span
    >
          <span id="textoBotao">Testar Conexão com o Servidor</span>
        </button>

        <div id="caixaAlerta" class="alert d-none mb-0" role="alert"></div>
      </div>
    </div>
  </div>
```

```
  <script>
    const btnVerificar = document.getElementById('btnVerificar');
    const iconeBotao = document.getElementById('iconeBotao');
    const spinnerCarregamento = document.getElementById('spinnerCarregamento');
    const textoBotao = document.getElementById('textoBotao');
    const caixaAlerta = document.getElementById('caixaAlerta');

    btnVerificar.addEventListener('click', async () => {
      // 1. Início: ativa o spinner e desabilita o botão
      btnVerificar.disabled = true;
      iconeBotao.classList.add('d-none');
      spinnerCarregamento.classList.remove('d-none');
      textoBotao.textContent = 'Consultando Servidor...';
      caixaAlerta.classList.add('d-none');

      try {
        const resposta = await fetch('http://127.0.0.1:5000/api/status');
        if (!resposta.ok) {
          throw new Error(`O servidor retornou o código de erro HTTP ${resposta.status}.`);
        }
        const dados = await resposta.json();

        caixaAlerta.className = 'alert alert-success mb-0';
        caixaAlerta.innerHTML = `
          <i class="bi bi-check-circle-fill me-1"></i>
          <strong>API Online!</strong> Conectado ao servidor (${dados.servidor}).
        `;
      } catch (erro) {
        caixaAlerta.className = 'alert alert-danger mb-0';
        caixaAlerta.innerHTML = `
          <i class="bi bi-exclamation-triangle-fill me-1"></i>
          <strong>Falha de Conectividade:</strong> Não foi possível falar com a API.<br>
          <small class="text-muted">Detalhe: ${erro.message}</small>
        `;
      } finally {
        // 4. O bloco FINALLY é SEMPRE executado, garantindo o fim do spinner
        spinnerCarregamento.classList.add('d-none');
        iconeBotao.classList.remove('d-none');
        textoBotao.textContent = 'Testar Conexão com o Servidor';
        btnVerificar.disabled = false;
        caixaAlerta.classList.remove('d-none');
      }
    });
  </script>
</body>
</html>
```

Como testar a Resiliência da Pílula 4

1. Com o servidor Flask ativo, clique no botão: observe o spinner piscar e o alerta verde de sucesso aparecer.

2. Em seguida, vá ao terminal do Python e encerre o servidor pressionando **Ctrl+C**.

3. Clique no botão novamente: note que o spinner gira durante a tentativa de conexão, é desligado com segurança pelo bloco `finally`, e um alerta vermelho amigável informa a falha sem quebrar a

aplicação!

Consolidação dos Conceitos e Transição para o Projeto Integrado

Com estes 4 fundamentos atômicos assimilados e testados individualmente (**Fetch GET**, **Flask com CORS**, **Fetch POST** e **Resiliência com Spinner/Finally**), avançaremos agora para a construção da **Aplicação Completa da Vitrine Comunitária Conectada**, integrando banco relacional **SQLite**, camadas modulares em **ES6 Modules** e contingência offline.

Implementação Passo a Passo Guiada

Seguiremos a sequência didática em 6 fases de implementação estruturadas. Todo o código fornecido é **100% completo, sem reticências ou omissões**.

Fase 1/6: Configuração do Backend em Flask & SQLite (backend/)

Responsabilidade Arquitetural: Criar o ambiente isolado Python (virtualenv), instalar o microframework Flask com suporte a CORS e implementar a API RESTful com conexão ao SQLite em arquivo (`vitrine.db`).

Passo 1.1: Criar o arquivo de dependências (`backend/requirements.txt`)

```
Flask>=3.0.0
flask-cors>=4.0.0
```

Passo 1.2: No terminal do VS Code, criar o ambiente virtual e instalar dependências:

```
# Acesse a pasta do backend
cd roteiro-5/exercicio-roteiro5/backend

# Cria o ambiente virtual isolado
python3 -m venv venv

# Ativa o ambiente virtual (Linux/Mac)
source venv/bin/activate
# No Windows PowerShell utilize: .\venv\Scripts\Activate.ps1

# Instala o Flask e o CORS
pip install -r requirements.txt
```

Passo 1.3: Criar o arquivo completo do servidor (`backend/app.py`):

```
import os
import sqlite3
from datetime import datetime
from flask import Flask, jsonify, request
from flask_cors import CORS

app = Flask(__name__)
# Habilita CORS para permitir que o Live Server (porta 5500) acesse o Flask (5000)
CORS(app)

DB_PATH = os.path.join(os.path.dirname(__file__), 'vitrine.db')

def obter_conexao():
    """Conecta ao SQLite retornando linhas como dicionários."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def inicializar_banco():
```

```
    """Cria a tabela servicos e popula com os 3 registros modelo de Garopaba se vazia."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS servicos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            categoria TEXT NOT NULL,
            bairro TEXT NOT NULL,
            precoBase REAL NOT NULL,
            telefone TEXT NOT NULL,
            descricao TEXT,
            dataCadastro TEXT
        )
    ''')

    cursor.execute('SELECT COUNT(*) AS total FROM servicos')
    if cursor.fetchone()['total'] == 0:
        dados_semente = [
            (
                'Maré Alta Artesanatos & Cerâmicas', 'Artesanato', 'Centro Histórico',
                35.00, '48991234567',
                'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.',
                datetime.now().isoformat()
            ),
            (
                'Garopaba Web & Design Studio', 'Tecnologia', 'Ferrugem',
                150.00, '48998765432',
                'Criação de websites profissionais responsivos, cardápios digitais e suporte para comércio
    local.',
                datetime.now().isoformat()
            ),
            (
                'Pescado Fresco do Zequinha', 'Alimentação', 'Canto das Canoas',
                42.00, '48984561234',
                'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e
    pontualidade.',
                datetime.now().isoformat()
            )
        ]
        cursor.executemany('''
            INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', dados_semente)
        conn.commit()
    conn.close()

@app.route('/api/status', methods=['GET'])
def verificar_status():
    """Diagnóstico de conectividade com a API."""
    return jsonify({
        'status': 'online',
        'mensagem': 'API REST da Vitrine de Garopaba operacional.',
        'banco': 'SQLite 3'
    }), 200

@app.route('/api/servicos', methods=['GET'])
def listar_servicos():
    """READ: Retorna a lista de todos os serviços ordenados pelo ID decrescente."""
    conn = obter_conexao()
    cursor = conn.cursor()
```

```
    cursor.execute('SELECT * FROM servicos ORDER BY id DESC')
    linhas = cursor.fetchall()
    conn.close()
    return jsonify([dict(l) for l in linhas]), 200

@app.route('/api/servicos', methods=['POST'])
def cadastrar_servico():
    """CREATE: Grava um novo serviço enviado em formato JSON no SQLite."""
    dados = request.get_json()
    if not dados:
        return jsonify({'erro': 'Payload inválido.'}), 400

    conn = obter_conexao()
    cursor = conn.cursor()
    data_atual = datetime.now().isoformat()

    cursor.execute('''
        INSERT INTO servicos (nome, categoria, bairro, precoBase, telefone, descricao, dataCadastro)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (
        dados['nome'].strip(),
        dados['categoria'].strip(),
        dados['bairro'].strip(),
        float(dados['precoBase']),
        dados['telefone'].strip(),
        dados.get('descricao', '').strip(),
        data_atual
    ))
    conn.commit()
    novo_id = cursor.lastrowid
    conn.close()

    return jsonify({
        'id': novo_id,
        'nome': dados['nome'].strip(),
        'categoria': dados['categoria'].strip(),
        'bairro': dados['bairro'].strip(),
        'precoBase': float(dados['precoBase']),
        'telefone': dados['telefone'].strip(),
        'descricao': dados.get('descricao', '').strip(),
        'dataCadastro': data_atual
    }), 201

@app.route('/api/servicos/<int:id>', methods=['DELETE'])
def excluir_servico(id):
    """DELETE: Exclui um serviço pelo ID."""
    conn = obter_conexao()
    cursor = conn.cursor()
    cursor.execute('DELETE FROM servicos WHERE id = ?', (id,))
    conn.commit()
    conn.close()
    return jsonify({'mensagem': f'Serviço {id} removido.'}), 200

if __name__ == '__main__':
    inicializar_banco()
    print('Servidor Flask ativo na porta 5000: http://127.0.0.1:5000')
    app.run(host='127.0.0.1', port=5000, debug=True)
```

Fase 2/6: Executando e Testando a API RESTful

Responsabilidade Arquitetural: Iniciar o servidor Flask na porta 5000 e validar os endpoints de saúde (`/api/status`) e listagem (`/api/servicos`) diretamente no terminal e navegador antes da integração.

Passo 2.1: Iniciar o servidor no terminal:

```
python app.py
```

Passo 2.2: Testar no navegador ou via cURL em outra aba do terminal:

```
curl http://127.0.0.1:5000/api/status
```

Resposta esperada do servidor (JSON com status 200):

```
{"banco":"SQLite 3","mensagem":"API REST da Vitrine de Garopaba operacional.","status":"online"}
```

Checkpoint de Validação da Fase 2 Abra no navegador `http://127.0.0.1:5000/api/servicos`. Você deverá ver a lista JSON com os 3 registros semente cadastrados automaticamente no arquivo `vitrine.db`.

Fase 3/6: A Revolução Assíncrona no vitrineService.js

Responsabilidade Arquitetural: Refatorar os métodos de leitura e escrita para utilizarem requisições HTTP assíncronas com `fetch()` e `async/await`, mantendo uma camada de contingência transparente com `localStorage`.

```
// js/services/vitrineService.js
const API_BASE_URL = 'http://127.0.0.1:5000/api';
const STORAGE_KEY = 'garopaba_vitrine_servicos';

let apiConectada = false;

export const DADOS_INICIAIS = [
  {
    id: '1',
    nome: 'Maré Alta Artesanatos & Cerâmicas',
    categoria: 'Artesanato',
    bairro: 'Centro Histórico',
    precoBase: 35.00,
    telefone: '48991234567',
    descricao: 'Peças artesanais e utilitárias modeladas à mão com argila local e conchas de Garopaba.'
  },
  {
    id: '2',
```

```
    nome: 'Garopaba Web & Design Studio',
    categoria: 'Tecnologia',
    bairro: 'Ferrugem',
    precoBase: 150.00,
    telefone: '48998765432',
    descricao: 'Criação de websites profissionais responsivos, cardápios digitais e suporte para comércio
    local.'
  },
  {
    id: '3',
    nome: 'Pescado Fresco do Zequinha',
    categoria: 'Alimentação',
    bairro: 'Canto das Canoas',
    precoBase: 42.00,
    telefone: '48984561234',
    descricao: 'Peixes frescos e frutos do mar da pesca artesanal diária entregues com higiene e pontualidade
    .'
  }
];

export function estaConectadoNaApi() {
  return apiConectada;
}

export async function testarConexaoApi() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/status`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(2000)
    });
    apiConectada = resposta.ok;
    return apiConectada;
  } catch (erro) {
    apiConectada = false;
    return false;
  }
}

// Rotinas Locais de Contingência (Fallback)
function obterServicosLocal() {
  const dados = localStorage.getItem(STORAGE_KEY);
  if (!dados) {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(DADOS_INICIAIS));
    return DADOS_INICIAIS;
  }
  try { return JSON.parse(dados); } catch (e) { return []; }
}

function salvarServicoLocal(novoServico) {
  const servicos = obterServicosLocal();
  const servicoCompleto = {
    id: Date.now().toString(),
    dataCadastro: new Date().toISOString(),
    ...novoServico
  };
  servicos.unshift(servicoCompleto);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
  return servicoCompleto;
}
```

```
function removerServicoLocal(id) {
  const servicos = obterServicosLocal();
  const filtrados = servicos.filter(s => s.id.toString() !== id.toString());
  localStorage.setItem(STORAGE_KEY, JSON.stringify(filtrados));
  return filtrados;
}

// Operações Assíncronas com Fetch API
export async function obterServicos() {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'GET',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(3000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const dados = await resposta.json();
    apiConectada = true;
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dados));
    return dados;
  } catch (erro) {
    apiConectada = false;
    console.warn('API Flask offline. Ativando contingência LocalStorage:', erro.message);
    return obterServicosLocal();
  }
}

export async function salvarServico(novoServico) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json'
      },
      body: JSON.stringify(novoServico),
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    const criado = await resposta.json();
    apiConectada = true;
    const servicos = obterServicosLocal();
    servicos.unshift(criado);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(servicos));
    return criado;
  } catch (erro) {
    apiConectada = false;
    console.warn('Falha no envio à API. Salvando localmente:', erro.message);
    return salvarServicoLocal(novoServico);
  }
}

export async function removerServico(id) {
  try {
    const resposta = await fetch(`${API_BASE_URL}/servicos/${id}`, {
      method: 'DELETE',
      headers: { 'Accept': 'application/json' },
      signal: AbortSignal.timeout(4000)
    });
    if (!resposta.ok) throw new Error(`HTTP ${resposta.status}`);
    apiConectada = true;
```

```
    removerServicoLocal(id);
    return true;
  } catch (erro) {
    apiConectada = false;
    console.warn('Falha na exclusão remota. Removendo localmente:', erro.message);
    removerServicoLocal(id);
    return true;
  }
}
```

Fase 4/6: Camada de Visão & Feedbacks de Rede (vitrineView.js)

Responsabilidade Arquitetural: Adicionar componentes visuais de carregamento (`exibirSpinner`) e indicador de conectividade em tempo real com a API no topo da página.

```
// js/views/vitrineView.js
import { formatarMoeda, formatarTelefone, escapeHtml } from '../utils/formatters.js';

const CORES_CATEGORIA = {
  'Alimentação': 'success',
  'Tecnologia': 'info',
  'Artesanato': 'warning',
  'Serviços Gerais': 'primary',
  'Turismo': 'secondary'
};

export function criarCardHtml(s) {
  const corBadge = CORES_CATEGORIA[s.categoria] || 'primary';
  const telNumeros = (s.telefone || '').replace(/\D/g, '');

  return `
    <div class="col-md-6 col-lg-4 mb-4">
      <div class="card h-100 shadow-sm border-0 vitrine-card rounded-4 overflow-hidden">
        <div class="card-body p-4 d-flex flex-column">
          <div class="d-flex justify-content-between align-items-center mb-2">
            <span class="badge bg-${corBadge} px-3 py-2 rounded-pill font-outfit">
              <i class="bi bi-tag-fill me-1"></i>${escapeHtml(s.categoria)}
            </span>
            <span class="fw-bold text-success font-outfit fs-5">
              ${formatarMoeda(s.precoBase)}
            </span>
          </div>

          <h5 class="card-title fw-bold text-dark mt-2 mb-1">${escapeHtml(s.nome)}</h5>
          <h6 class="text-muted small mb-3">
            <i class="bi bi-geo-alt-fill text-danger me-1"></i>${escapeHtml(s.bairro)}
          </h6>
          <p class="card-text text-secondary small flex-grow-1" style="line-height: 1.5;">
            ${escapeHtml(s.descricao)}
          </p>

          <hr class="my-3 text-muted opacity-25">

          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a href="https://wa.me/55${telNumeros}" target="_blank" class="btn btn-sm btn-outline-success
    rounded-pill px-3 fw-bold">
              <i class="bi bi-whatsapp me-1"></i>${formatarTelefone(s.telefone)}
            </a>
            <div class="d-flex gap-1">
              <button class="btn btn-sm btn-outline-danger btn-excluir rounded-pill px-2" data-id="${s.id}"
    title="Remover da vitrine">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          </div>
        </div>
      </div>
```

```
    </div>
  `;
}

export function renderizarCards(servicos, containerElement) {
  if (!containerElement) return;
  if (!servicos || servicos.length === 0) {
    containerElement.innerHTML = `
      <div class="col-12 py-5 text-center text-muted">
        <i class="bi bi-inbox fs-1 d-block mb-3 text-secondary opacity-50"></i>
        <h5 class="fw-bold">Nenhum serviço encontrado</h5>
        <p class="small">Cadastre um novo serviço ou altere o filtro de categoria.</p>
      </div>
    `;
    return;
  }
  containerElement.innerHTML = servicos.map(s => criarCardHtml(s)).join('');
}

export function exibirSpinner(containerElement) {
  if (!containerElement) return;
  containerElement.innerHTML = `
    <div class="col-12 py-5 text-center text-primary">
      <div class="spinner-border text-primary" role="status" style="width: 3rem; height: 3rem;">
        <span class="visually-hidden">Carregando dados da API...</span>
      </div>
      <p class="mt-3 text-muted small fw-semibold">Consultando serviços no backend Flask (SQLite)...</p>
    </div>
  `;
}

export function atualizarStatusConexao(isOnline, statusBadgeEl) {
  if (!statusBadgeEl) return;
  if (isOnline) {
    statusBadgeEl.className = 'badge bg-success-subtle text-success border border-success-subtle px-3 py-2
     rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-cloud-check-fill me-1"></i> Backend Online (Flask + SQLite)';
  } else {
    statusBadgeEl.className = 'badge bg-warning-subtle text-warning border border-warning-subtle px-3 py-2
     rounded-pill fw-semibold';
    statusBadgeEl.innerHTML = '<i class="bi bi-hdd-network-fill me-1"></i> Modo Offline (LocalStorage)';
  }
}

export function exibirToast(mensagem, tipo = 'success') {
  const toastEl = document.getElementById('toastNotificacao');
  const toastMsgEl = document.getElementById('toastMensagem');
  if (!toastEl || !toastMsgEl) return;
  toastEl.className = `toast align-items-center text-white bg-${tipo} border-0 shadow-lg`;
  toastMsgEl.innerText = mensagem;
  const bsToast = bootstrap.Toast.getOrCreateInstance(toastEl, { delay: 3500 });
  bsToast.show();
}
```

Fase 5/6: Orquestrador Assíncrono da Aplicação (main.js)

Responsabilidade Arquitetural: Coordenar os fluxos de eventos da interface utilizando `await` para aguardar a persistência no banco remoto antes de fechar modais e atualizar a vitrine.

```
// js/main.js
import {
  obterServicos,
  salvarServico,
  removerServico,
  testarConexaoApi,
  estaConectadoNaApi
} from './services/vitrineService.js';

import {
  renderizarCards,
  exibirSpinner,
  atualizarStatusConexao,
  exibirToast
} from './views/vitrineView.js';

document.addEventListener('DOMContentLoaded', async () => {
  const form = document.getElementById('formCadastro');
  const container = document.getElementById('vitrineContainer');
  const filtroCategoria = document.getElementById('filtroCategoria');
  const contadorEl = document.getElementById('totalServicosBadge');
  const statusApiBadge = document.getElementById('statusApiBadge');
  const modalEl = document.getElementById('modalCadastro');
  const btnSalvarTexto = document.getElementById('btnSalvarTexto');
  const btnSalvar = document.getElementById('btnSalvar');

  let cacheServicosAtuais = [];

  async function atualizarInterface() {
    exibirSpinner(container);
    const conectado = await testarConexaoApi();
    atualizarStatusConexao(conectado, statusApiBadge);

    cacheServicosAtuais = await obterServicos();
    const categoriaSelecionada = filtroCategoria ? filtroCategoria.value : 'todas';
    const servicosFiltrados = categoriaSelecionada === 'todas'
      ? cacheServicosAtuais
      : cacheServicosAtuais.filter(s => s.categoria === categoriaSelecionada);

    renderizarCards(servicosFiltrados, container);
    if (contadorEl) {
      contadorEl.innerText = `${cacheServicosAtuais.length} Serviços Cadastrados`;
    }
  }

  if (filtroCategoria) {
    filtroCategoria.addEventListener('change', () => {
      const categoria = filtroCategoria.value;
      const filtrados = categoria === 'todas'
        ? cacheServicosAtuais
        : cacheServicosAtuais.filter(s => s.categoria === categoria);
      renderizarCards(filtrados, container);
```

```
  });
}

// Inicialização assíncrona
await atualizarInterface();

// Reset do formulário ao abrir modal
if (modalEl) {
  modalEl.addEventListener('show.bs.modal', () => {
    form.reset();
    form.classList.remove('was-validated');
  });
}

// Cadastro de Novo Serviço via HTTP POST
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (!form.checkValidity()) {
    e.stopPropagation();
    form.classList.add('was-validated');
    return;
  }

  const getVal = (id) => document.getElementById(id).value.trim();
  const dados = {
    nome: getVal('nome'),
    categoria: document.getElementById('categoria').value,
    bairro: getVal('bairro'),
    precoBase: parseFloat(getVal('precoBase')) || 0,
    telefone: getVal('telefone'),
    descricao: getVal('descricao')
  };

  if (btnSalvar) btnSalvar.disabled = true;
  if (btnSalvarTexto) btnSalvarTexto.innerText = 'Gravando...';

  try {
    await salvarServico(dados);
    const msg = estaConectadoNaApi()
      ? 'Serviço gravado no banco de dados SQLite!'
      : 'Serviço salvo localmente (Modo Offline).';
    exibirToast(msg, 'success');

    const modal = bootstrap.Modal.getOrCreateInstance(modalEl);
    modal.hide();
    await atualizarInterface();
  } catch (erro) {
    exibirToast('Erro ao processar cadastro.', 'danger');
  } finally {
    if (btnSalvar) btnSalvar.disabled = false;
    if (btnSalvarTexto) btnSalvarTexto.innerText = 'Salvar no Banco';
  }
});

// Exclusão Assíncrona via HTTP DELETE
container.addEventListener('click', async (e) => {
  const btnExcluir = e.target.closest('.btn-excluir');
  if (btnExcluir) {
    const id = btnExcluir.getAttribute('data-id');
    if (confirm('Deseja realmente remover este serviço da vitrine?')) {
      await removerServico(id);
```

```
        await atualizarInterface();
        exibirToast('Serviço excluído da vitrine.', 'warning');
      }
    }
  });
});
```

Fase 6/6: Verificação de Integração & Prova de Contingência

Responsabilidade Arquitetural: Executar o roteiro de testes em dois cenários: com a API online no laboratório e com o servidor desligado comprovando a resiliência offline.

1. Teste Conectado (API Online):

- Inicie o backend com `python app.py` na porta `5000`;
- Abra o frontend via Live Server (`http://127.0.0.1:5500/index.html`);
- O badge no topo deve exibir: ***Backend Online (Flask + SQLite)***;
- Cadastre um novo serviço e observe a requisição HTTP POST sendo logada no terminal do Flask!

2. Teste de Resiliência (Fallback LocalStorage):

- No terminal do Flask, pressione `Ctrl + C` para desligar o servidor;
- Recarregue a página no navegador com `F5`;
- O frontend detecta a indisponibilidade e muda o badge para: ***Modo Offline (LocalStorage)***;
- Os cards continuam aparecendo normalmente a partir da cópia em cache!

Critério de Conclusão do Roteiro 05 A aplicação demonstrou sucesso na comunicação assíncrona cliente-servidor via Fetch API, realizou persistência no SQLite e provou resiliência arquitetural ao operar em modo de contingência offline.

Quadro Comparativo: Evolução do Roteiro 04 para o Roteiro 05

Componente	Roteiro 04 (Módulo 2)	Roteiro 05 (Módulo 3)
Ambiente de Dados	LocalStorage (isolado na aba do navegador).	Banco relacional SQLite (`vitrine.db`) no Flask.
Comunicação	Chamadas de função síncronas em memória.	Chamadas assíncronas de rede com `fetch()` e `await`.
Segurança de Rede	Inexistente (tudo no cliente).	Configuração de cabeçalhos CORS (`flask-cors`).
Experiência (UX)	Resposta imediata sem feedback de rede.	Spinners de espera e badges de status de conectividade.
formatters.js	Funções puras de formatação.	**100% idêntico!** Prova de desacoplamento limpo.

Parabéns! Você implementou uma arquitetura moderna full-stack desacoplada, apta para o ecossistema profissional e pronta para avançar rumo às Single Page Applications (SPAs) e ao projeto de extensão da SNCT!